LESTARI BOT : LESTARI DORMITORY SYSTEM

SUFIANA ADLIN BINTI BAHAROM

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

TABLE OF CONTENT

CHAPTER 1. INTRODUCTION

1.1 Introduction

The Lestari Dormitory System, integrated with the sophisticated "LesBot" artificial intelligence engine, was engineered to provide residents and administrative staff at Universiti Teknikal Malaysia Melaka (UTeM) with a seamless, highly available, and centralized platform for managing residential operations. Recognizing the systemic inefficiencies of traditional facility management such as restricted physical office hours, fragmented communication channels like WhatsApp, and vulnerable paper-based logbooks LesBot is designed to digitize, simplify, and drastically enhance the entire dormitory administration lifecycle. This application seeks to streamline the complex logistics of tracking student room histories, delegating maintenance requests, and managing financial penalty ledgers, guaranteeing that both students and management have reliable, transparent access to a secure digital ecosystem.

By utilizing cutting-edge Generative Artificial Intelligence (Google Gemini 1.5 Flash) and enforcing a strict Third Normal Form (3NF) relational database architecture, LesBot tackles the operational bottlenecks that campus staff encounter daily. Whether a student needs to report an urgent plumbing malfunction at midnight, requires bilingual guidance on residential policies, or needs to securely review their outstanding penalty ledgers, LesBot provides an asynchronous, 24/7 reliable solution that alleviates the stress of institutional bureaucracy. This level of continuous availability ensures that critical facility data is never lost, delayed, or mismanaged due to human error or restricted weekday office hours.

To ensure a flawless user experience and uncompromised data integrity, the system offers several advanced key features tailored to specific administrative roles. Students can effortlessly interact with the multi-lingual LesBot chatbot to receive instant troubleshooting or be automatically redirected to submit standardized maintenance tickets. Staff members and technicians are provided with a dedicated operational dashboard to track, accept, and resolve active field tasks with precision. Meanwhile, high-level Administrators are equipped with enterprise-grade oversight tools, including automated accountability through a forensic system audit trail, secure soft-delete archiving protocols, and role-based access controls. By securely isolating these workflows and utilizing atomic database transactions, LesBot promotes a sense of operational tranquility, knowing that every action is meticulously recorded and shielded from data corruption.

In summary, the Lestari Dormitory System (LesBot) is a comprehensive, forward-thinking solution that revolutionizes the way university residential infrastructure is managed. By transitioning from a reactive, manual paradigm to a proactive, AI-driven digital environment, LesBot provides an essential service that dramatically enhances both the student living experience and the operational excellence of the university staff. This initiative not only stands as a rigorous application of advanced software engineering and database administration principles, but it also directly reflects the university’s commitment to building a modern "Smart Campus," making it an invaluable addition to UTeM’s digital infrastructure.

1.2 Problem statement(s)

- **Structural Inefficiency of Manual Reporting**: Students must physically visit the office or wait for office hours to report urgent issues like water leakages or electrical failures.

- **Data Fragmentation and Integrity:** Relying on WhatsApp or Google Forms results in "orphaned data" that isn't linked to a central database, making it impossible to track recurring maintenance trends.

- **Information Bottleneck**: Staff are overwhelmed by repetitive FAQs, delaying their ability to handle critical on-site repairs.

1.3 Objective

- **To Design** a robust, normalized relational database using MySQL to centralize student, staff, maintenance, and penalty records.

- **To Develop** a responsive web application that facilitates seamless CRUD (Create, Read, Update, Delete) operations for different user roles.

- **To Integrate** a Natural Language Processing (NLP) chatbot session to automate initial troubleshooting and ticket categorization.

- **To Validate** the system's usability and database performance through comprehensive Black-box and User Acceptance Testing (UAT).

1.4 Scope

- **Target Audience**: Residents of UTeM hostels, hostel technicians, and administrative wardens.

- **Functional Scope**: Chatbot interaction logs, automated ticketing, penalty dashboard, and administrative reporting.

- **Technical Scope**: Implementation using the XAMPP stack (Apache, MySQL, PHP/Node.js) and Bootstrap for responsive design.

1.5 Project Significance

The implementation of the Lestari Dormitory System (LesBot) represents a significant paradigm shift in how Universiti Teknikal Malaysia Melaka (UTeM) manages its residential infrastructure, offering profound multidimensional benefits to students, administrative staff, and the institution at large.

Primarily, the system radically improves the quality of student life by dismantling the traditional 10:00 AM to 5:00 PM administrative bottleneck. By providing a streamlined, asynchronous platform integrated with a 24/7 Generative AI helpdesk, it eliminates the stress and logistical friction students face when attempting to report urgent maintenance hardware failures, track room histories, or query financial penalties. This ubiquitous accessibility ensures that students are supported around the clock,

allowing them to focus entirely on their academic pursuits without being hindered by bureaucratic delays.

From an administrative and operational perspective, LesBot revolutionizes staff workflow. By offloading repetitive, high-volume inquiries to the multi-lingual Gemini AI engine, the system dramatically reduces cognitive overload for dormitory management and technicians. This intelligent triage allows human staff to allocate their time and resources exclusively to critical, hands-on field operations—thereby increasing the overall utilization, response time, and efficiency of campus maintenance services.

Furthermore, as a centralized digital ecosystem, the project fundamentally secures institutional data. By replacing vulnerable physical logbooks and fragmented WhatsApp communications with a strict, Third Normal Form (3NF) Azure cloud database, the system eradicates the risks of data loss, orphaned records, and financial discrepancies. The inclusion of cryptographic access controls, soft-delete archiving protocols, and a forensic system audit trail ensures total accountability and operational transparency, safeguarding both student privacy and institutional ledgers.

Ultimately, LesBot is more than a functional utility; it serves as a robust prototype for UTeM's "Smart Campus" initiative. It showcases the university's proactive commitment to leveraging enterprise-grade cloud architecture and cutting-edge artificial intelligence to modernize student welfare. This successful digital transformation not only enhances the day-to-day operational excellence of the university's housing infrastructure but significantly elevates the institution's reputation as a forward-thinking, technologically advanced academic environment.

1.6 Expected Output

The project anticipates several tangible deliverables upon completion:

- **Functional Web-Based Platform**: A fully hosted web application accessible via desktop and mobile browsers, ensuring students can report issues from anywhere within the campus.

- **Intelligent Chatbot Interface**: A 24/7 virtual assistant capable of resolving non-complex queries and autonomously creating maintenance tickets in the database without human intervention.

- **Centralized Relational Database**: A professionally designed MySQL backend that ensures high data integrity, minimized redundancy, and structured logging of all residential interactions.

- **Comprehensive Project Report**: A formal PSM/FYP document detailing the system architecture, database normalization (3NF), and test results, meeting the academic requirements for a BITD degree.

1.7 Conclusion

Chapter 1 has established the foundational framework for the Lestari-Bot project. By identifying the critical gaps in current manual dormitory processes specifically the lack of data integrity and accessibility that this project proposes a modern, AI-integrated solution. The objectives and scope defined here serve as the roadmap for the subsequent phases of analysis, design, and implementation. The successful deployment of this system is expected to significantly improve operational efficiency and student satisfaction within the university residential ecosystem.

CHAPTER 2. PROJECT METHODOLOGY AND PLANNING

2.1 Introduction

The success of a complex database-driven system like Lestari-Bot relies heavily on a structured development lifecycle. This chapter outlines the strategic approach taken to transition from the initial Workshop 1 C++ logic to a full-scale web application. It discusses the selection of the Agile Scrum Methodology, which is designed to handle the iterative nature of chatbot development and database normalization. Furthermore, this chapter provides a detailed roadmap of the project's schedule, ensuring that all academic milestones set by the faculty are met with technical precision.

In this chapter, a comprehensive exploration of the theoretical framework and developmental context underpinning the Lestari Dormitory System (LesBot) is presented. This section delves into the current literature surrounding smart campus residential infrastructure, centralized facility management solutions, and the transformative integration of Generative Artificial Intelligence—specifically Large Language Models (LLMs) such as Google Gemini—into institutional administrative workflows. By discussing pertinent facts and empirical findings from contemporary academic studies, this chapter critically analyzes existing traditional systems, such as paper-based physical logbooks and fragmented communication channels like WhatsApp. Through this comparative analysis, the critical operational gaps and bottlenecks inherent in the current manual management of student residences are identified, validating the necessity of the LesBot digital ecosystem tailored specifically for the Universiti Teknikal Malaysia Melaka (UTeM) student and staff community.

Furthermore, this chapter outlines the rigorous systematic approach and software engineering principles adopted during the lifecycle of the project. It details the specific Project Methodology utilized to architect the system's robust, Third Normal Form (3NF) relational database and interactive, role-based user interface. In addition to the developmental framework, a definitive breakdown of the required hardware specifications and advanced software environments—encompassing the deployment of Object-Oriented PHP, Azure Cloud MySQL Database infrastructure, and RESTful API neural integrations—is systematically documented. Finally, a meticulously structured project schedule and lifecycle milestones will be presented to illustrate the project's progression. Together, this comprehensive theoretical and methodological review establishes the critical foundation required to understand both the academic significance and the technical implementation of the LesBot 24/7 helpdesk and dormitory management platform.

2.2 Project Methodology

For the implementation of Lestari-Bot, the Agile Scrum Methodology is adopted. Unlike the traditional Waterfall model, which is rigid and sequential, Agile is highly iterative and well-suited for projects involving Natural Language Processing (NLP) and dynamic database schemas.

- **Iterative Development (Sprints):** The project is decomposed into two-week "Sprints." This allows for the core database architecture to be built and tested before the chatbot session logic is fully integrated.

- **Database Refinement:** As a BITD project, the database is the priority. Agile allows us to refine the 3rd Normal Form (3NF) schema during the development of the chatbot, ensuring that if new data requirements arise during bot testing (e.g., adding a sentiment_score field), they can be implemented without restarting the entire project.

- **Continuous Feedback:** By using an iterative approach, the User Interface (UI) can be adjusted based on early testing of the chatbot session, ensuring the final web portal is both responsive and user-friendly.

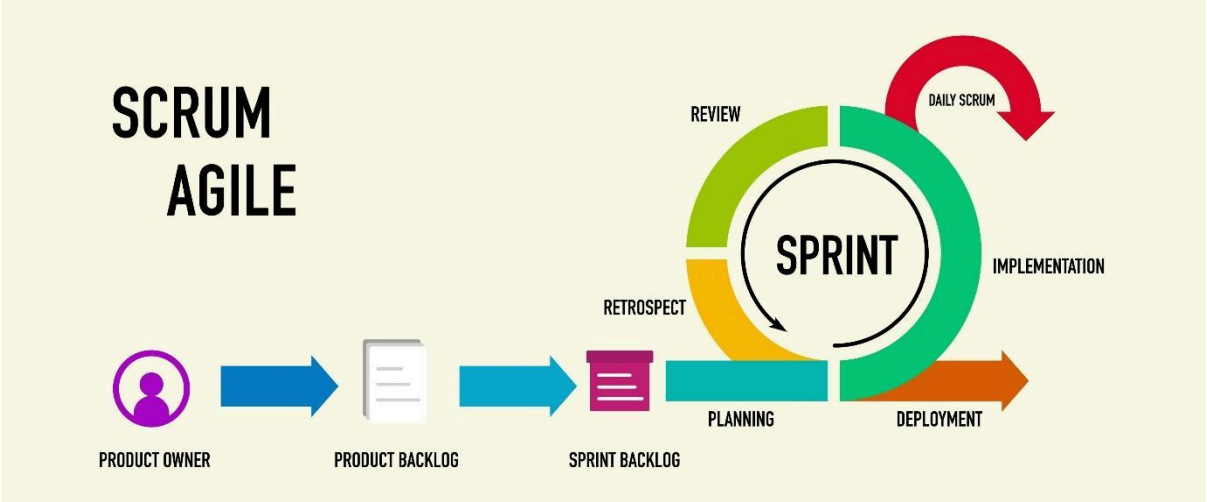


Figure 1 Serum Agile Methodology

The provided image is a visualization of the Scrum Agile framework, an iterative and incremental approach to managing and developing products. It showcases the key roles, artifacts, and events within a Scrum process, highlighting the cyclical nature of development sprints. The visual flow begins with the Product Owner and progresses through various stages, culminating in a continuous cycle of product development and improvement.

2.3 Project Schedule and Milestones

In alignment with the UTeM PSM 1 Milestones for Semester 2 2025/2026, the project is scheduled across four critical phases. Each phase is designed to ensure that the database integrity and system functionality are optimized before final submission.

MONTHS	16 – 20/3	23 – 27/3	30 - 3/4	6 – 10/4	13 – 17/4	20 – 24/4	27 -1/5	4 – 8/5	11 – 15/5	18 – 22/5	25 – 29/5	1 – 12/6		15 – 19/6	22 – 26/6
TOPIC	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Preliminary Fact-Finding and User Interview Preparation.															
Systematic Requirement Gathering (SRS) and Analysis of "As-Is" system gaps.															
System Requirement Specification (SRS) Documentation and Approval.															
Conceptual Database Design: Developing the initial Entity Relationship Diagram (ERD).															
Logical Database Design: Schema Mapping and Attribute Definition.															
Database Normalization Phase I: Achieving 1NF and 2NF (Removing Redundancy).															
Database Normalization Phase II: Achieving 3NF (Ensuring Referential Integrity).															

Backend Environment Setup: Configuring the XAMPP Server and Node.js/PHP Environment.															
Chatbot NLP Logic Development: Pattern Matching and Intent Classification.															
Integration Layer: Connecting Chatbot API to the MySQL Database.															
Frontend Development: Designing Responsive UI using Bootstrap 5.															
System Testing: Executing Black-box and White-box Test Cases.															
User Acceptance Testing (UAT) and System Debugging.															

2.4 Conclusion

Chapter 2 has detailed the methodological framework and the planning schedule that governs the development of Lestari-Bot. By selecting the Agile Scrum Methodology, the project gains the flexibility required to refine the complex relationship between the chatbot's conversational logic and the MySQL relational database. The established milestones provide a clear timeline for the transition from conceptual design to a fully functional, web-based management system. With the methodology and schedule firmly in place, the project proceeds to Chapter 3, which focuses on the in-depth analysis of the system's requirements and the current challenges in dormitory management.

CHAPTER 3. ANALYSIS

3.1 Introduction

The analysis phase is the cornerstone of the system development life cycle (SDLC). It serves as a bridge between the initial problem identification and the technical design. This chapter provides a granular evaluation of the existing dormitory management challenges through the PIECES Framework. By decomposing the system into performance, information, and service metrics, the project establishes a data-driven justification for the proposed web-based chatbot integration. Furthermore, this chapter defines the functional and non-functional requirements that will govern the database architecture and user interaction logic.

3.2 Problem Analysis

The current manual and console-based systems used for dormitory management at UTeM were analysed using the PIECES Framework. This analysis identifies the critical bottlenecks that Lestari-Bot aims to resolve.

	Analysis of Current "As-Is" System	**Proposed "To-Be" System (Lestari-Bot)**
Performance	Slow response times; reports are often buried in WhatsApp chats or physical logbooks, requiring manual sorting.	High-speed processing; the chatbot categorizes tickets instantly using indexed MySQL lookups.
Information	Data is fragmented and redundant. There is no relational link between a student's penalty history and their maintenance requests.	Centralized relational database ensures data integrity and provides a "360-degree" view of student residential records.
Economics	High administrative costs due to staff spending significant hours answering repetitive FAQs regarding hostel rules.	Cost-efficient; the automated chatbot session handles approximately 70% of routine inquiries without human intervention.
Control	Weak security protocols; physical logs can be lost, and digital spreadsheets lack role-based access control (RBAC).	Robust security; implemented via PHP Data Objects (PDO) to prevent SQL injection and strict RBAC for data privacy.
Efficiency	Redundant data entry; students must provide the same personal details every time they file a new report.	Optimized workflow; student data is pulled automatically from the tbl_students table during the chat session.
Service	Limited to office hours (8:00 AM – 5:00 PM). Issues reported on weekends remain unaddressed until Monday.	24/7 Availability; the web-based bot ensures students can log critical failures at any time with instant confirmation.

3.2.1. Flowchart is used to analyse the problem in the current system.

Each process and problem will be described below:

3.2.1.1. Flowchart for student Reporting (Current Manual Process – Face to Face)

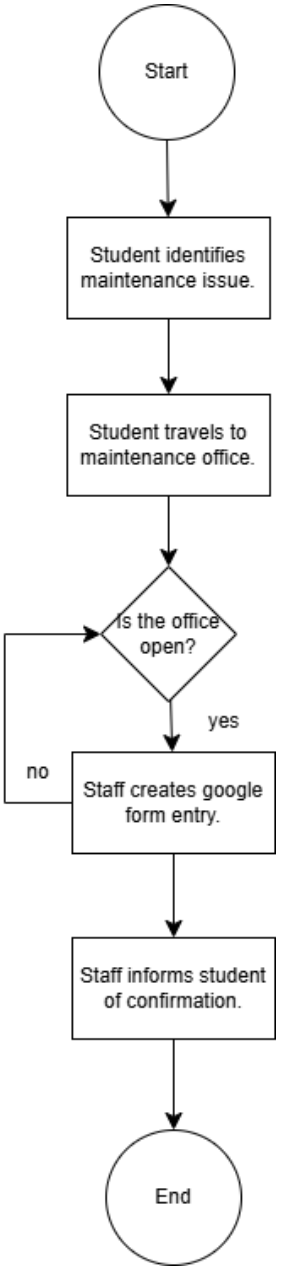


Figure 3.2.1.1.: Flowchart for student reporting (face to face)

Figure 3.2.1.1 illustrates the process of students reporting maintenance issues by physically going to the maintenance office. The process begins when a student identifies a maintenance issue and travels to the office. The key step involves determining if the office is open; if not, the student must return during operating hours, adding significant inconvenience. If the office is open, the student describes the issue to a staff member, who then creates a Google Form entry based on the student's description. Finally, the staff member informs the student that their report has been submitted. However, this

method presents several drawbacks. The entire process is 15 heavily dependent on the office's operating hours, rendering it inaccessible outside those times. The need to physically travel to the office introduces significant inconvenience and potential delays. Furthermore, the reliance on verbal communication between the student and staff member can lead to misinterpretations or the omission of crucial details. The staff member's accurate and consistent data entry into the Google Form is also a point of potential error. Finally, the student receives only a verbal confirmation, lacking any means to track the request or verify its accurate submission.

3.2.1.2. Flowchart for student Reporting (Current Manual Process - WhatsApp)

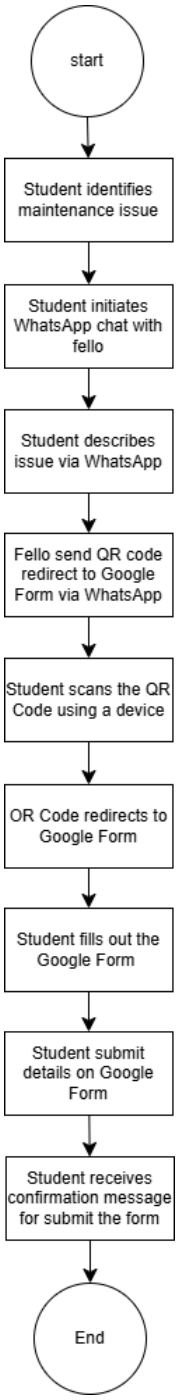


Figure 3.2.1.2.: Flowchart for student reporting (WhatsApp)

Figure 3.2.1.2 illustrates students reporting issues via WhatsApp to a 'Fellow', who provides a QR code for a Google Form. The student fills out this form - detailing their name, matric number, room, and the issue - and submits it. Key drawbacks include the dependency on 'Fello' availability, and potential delays related to Fellow responsiveness. Furthermore, the initial WhatsApp exchange and QR code are vulnerable to data loss from phone/WhatsApp issues, requiring the student to restart. Describing the issue via unstructured WhatsApp messages can lead to misinterpretations requiring re-entry on the Google Form. The Google Form also introduces risks of data entry errors and lacks real-time tracking.

Based on the analysis of the issues present in the current system, the main problems of the Lestari Bot: Lestari Dormitory System can be categorized into three key areas, as illustrated in Figure 2.3.2. These problems are elaborated as follows:

3.2.2. Problem Description

The main problems of Lestari Bot: Lestari Dormitory System are

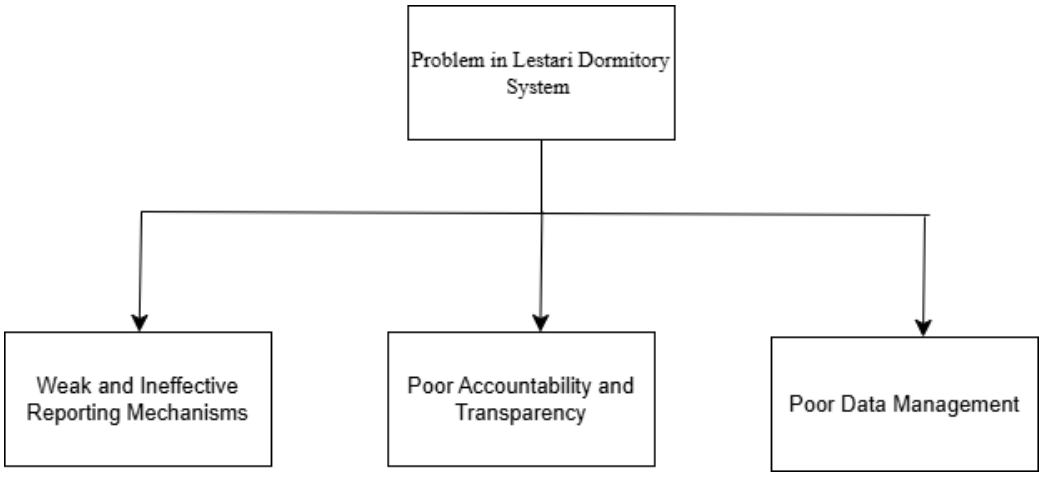


Figure 3.2.2.: Problem Description Structure Chart

a) Weak and ineffective reporting mechanisms.

The current reporting mechanisms for maintenance issues in the Lestari Bot: Lestari Dormitory System are weak and ineffective. This is primarily due to the reliance on manual procedures and basic technologies like Google Forms, which results in chaotic and inefficient reporting processes. The lack of standardization in reporting methods, such as using paper forms and emails, leads to inconsistent data collection and management. Furthermore, these manual processes often cause delays in both submitting and processing maintenance requests. This problem highlights the challenge faced by the system in efficiently managing maintenance concerns due to the disorganized nature of reporting. It underscores the need for a solution that allows for standardized, efficient, and timely reporting of maintenance issues to ensure that all necessary information is accurately collected and processed, thereby enhancing the overall effectiveness of the maintenance system.

b) Poor accountability and transparency.

The Lestari Bot: Lestari Dormitory System faces issues with poor accountability and transparency. Students lack real-time updates on the status of their maintenance requests, leading to frustration and mistrust. This limited visibility into the progress of their requests exacerbates the problem, as students are left without clear information on when issues will be resolved. Additionally, the system lacks an adequate feedback mechanism, limiting students' opportunities to provide input on maintenance services. This hinders continuous improvement, as valuable insights from users are not effectively captured or utilized. This problem highlights the challenge faced by the system in maintaining trust and ensuring that maintenance services meet student needs efficiently. It underscores the need for a solution that provides real-time updates and facilitates meaningful feedback to enhance accountability and transparency within the maintenance process.

c) Poor data management.

The Lestari Bot: Lestari Dormitory System struggles with poor data management. Maintenance data is disorganized, scattered across various platforms or stored manually, which complicates analysis and strategic planning. This disorganization hinders the ability to leverage data for informed decision-making. Furthermore, the lack of a centralized data management system prevents administrators from easily identifying maintenance trends or optimizing resource allocation. This problem highlights the challenge faced by the system in effectively utilizing data to improve maintenance operations. It underscores the need for a solution that centralizes data management, providing clear insights and enabling administrators to make data-driven decisions to enhance efficiency and resource utilization within the maintenance process.

The Lestari Bot: Lestari Dormitory System faces challenges in maintenance management due to inefficient manual reporting, lack of real-time updates and feedback, and disorganized data storage. These issues cause delays, reduce transparency, and hinder effective decision-making. A centralized, standardized, and data-driven solution is needed to improve reporting accuracy, accountability, and overall system efficiency.

3.3 The proposed improvements/solutions

The proposed solution, Lestari-Bot, is a comprehensive web-based platform that replaces the localized, command-line interface of the previous Workshop 1 system. This transition is not merely a change in interface but a fundamental re-engineering of the dormitory management workflow. By centralizing operations into a responsive web portal, the system ensures real-time data synchronization and accessibility.

- **Conversational AI Layer:** Instead of a static form, the system features a Chatbot Session that uses keyword-based NLP to help students "troubleshoot" before filing a report (e.g., "Check if your room switch is on before reporting a power trip").

- **Automated Ticket Dispatch:** Once a bot identifies a valid maintenance issue, it automatically assigns a Severity_Level and updates the tbl_maintenance table, triggering

a notification on the technician's dashboard.

- **Integrated Penalty Dashboard:** A digitized module that allows students to view and verify penalties in real-time, reducing disputes between residents and management.

3.3.1 Proposed System Diagram

To illustrate the operational logic of Lestari-Bot, the following flowchart depicts the journey of a student from the initial inquiry to the database update. This diagram highlights the decision-making process within the Chatbot Session.

The design of the programs of the proposed system are illustrated by the flowcharts from Figure 2.3.1 to Figure

3.3.1. Flowchart of Lestari Dormitory System for Student.

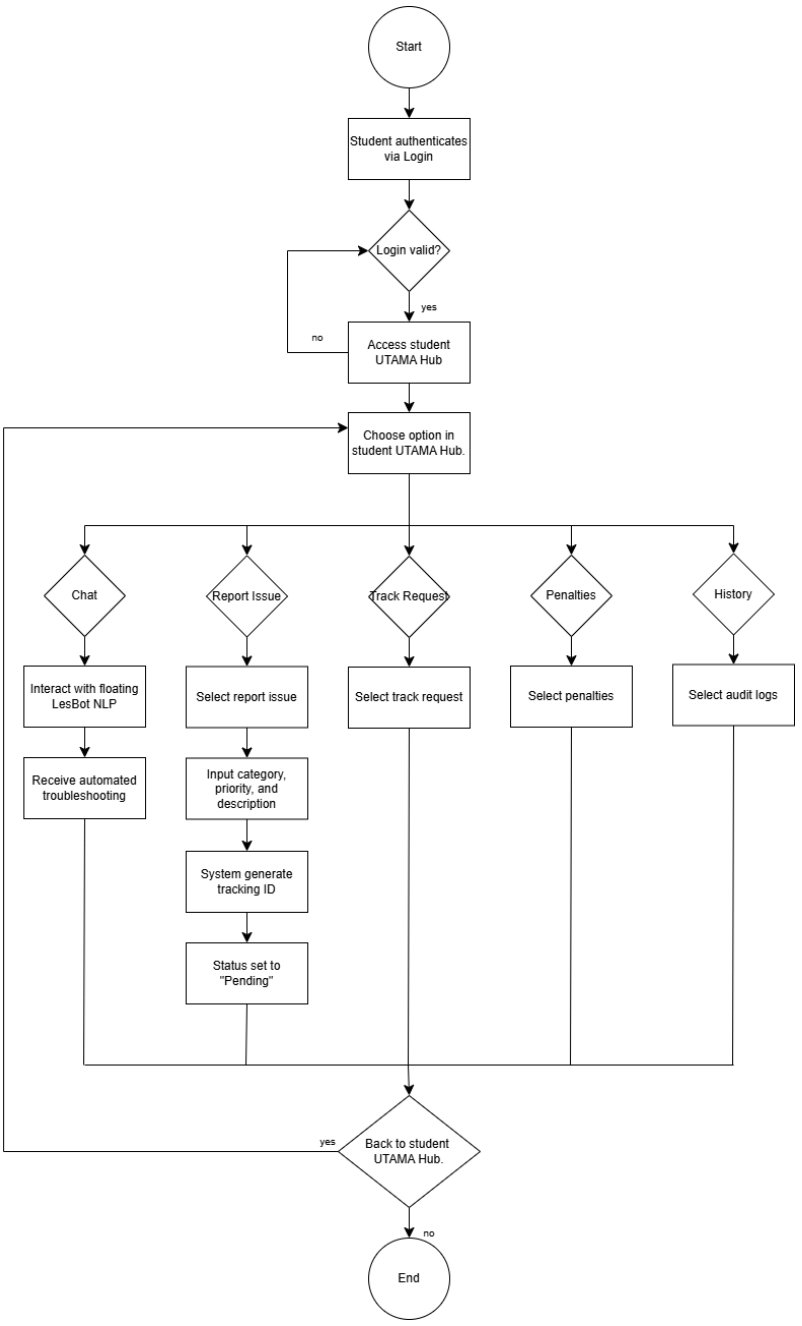


Figure 3.3.1.: Flowchart of Lestari Dormitory System for Student.

Figure 3.3.1 systematically illustrates the operational flowchart for the Student Module within the Lestari Dormitory System (LesBot). The procedural flow initiates with a secure authentication gate, where a student must input their credentials to access the system. A strict validation loop is enforced at this stage; if the authentication fails, the user is continuously redirected to the login interface until valid credentials are provided. Upon successful verification, the system grants secure access to the "student UTAMA Hub," which serves as the primary command dashboard for the user's session.

From this central hub, the system architecture branches into five distinct functional pathways, allowing the student to choose specific operations. If the student selects the "Chat" protocol, they directly interface with the integrated floating LesBot NLP (Natural Language Processing) engine to

receive immediate, automated troubleshooting. Should the user require physical facility intervention, they navigate to the "Report Issue" module. Here, the student is required to input specific parameters—namely the issue category, priority level, and a detailed description. Upon submission, the database autonomously generates a unique tracking ID and initializes the operational status to "Pending".

Alternatively, the student can access three targeted read-only modules: "Track Request" to monitor the real-time lifecycle of their active maintenance tickets, "Penalties" to review any outstanding financial or disciplinary ledgers, and "History" to pull records from the system's audit logs. Following the execution of any selected operation, the system presents a logical decision node evaluating whether the user wishes to navigate "Back to student UTAMA Hub". Selecting "yes" systematically loops the user back to the primary dashboard to perform additional tasks, while selecting "no" successfully terminates the operation, guiding the user to the "End" state of the session. This highly structured, modular design ensures that students experience a frictionless, intuitive user journey while maintaining strict session integrity throughout the web application.

3.3.2. Flowchart of Lestari Dormitory System for Staff

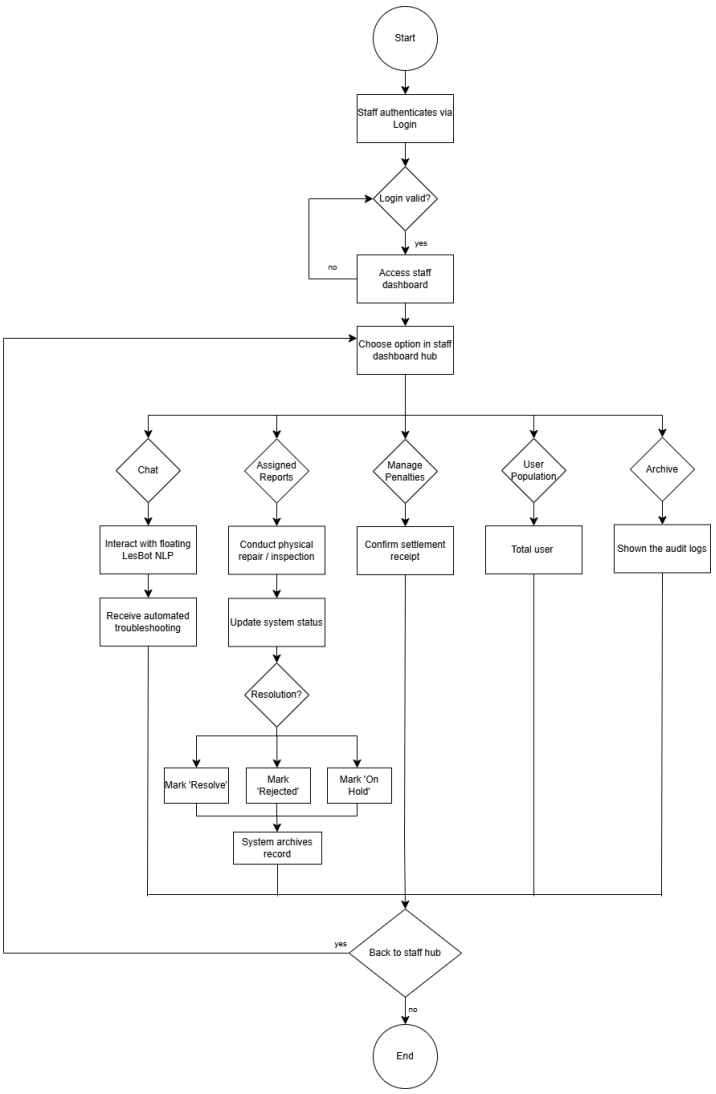


Figure 3.3.2.: Flowchart of Lestari Dormitory System for Staff

Figure 3.3.2 systematically delineates the operational architecture and procedural workflow for the Staff and Technician module within the Lestari Dormitory System (LesBot). The sequence commences with a strict security gateway, where technical personnel must authenticate their credentials via the login interface. A rigorous validation loop is enforced at this juncture; if unauthorized or incorrect credentials are submitted, the system continuously redirects the user to the login screen, thereby securing the system against unauthorized entry. Upon successful authentication, the staff member is securely routed to the primary staff dashboard, which serves as the centralized tactical command hub for their designated facility operations.

Once inside the dashboard hub, the system architecture branches into five specialized operational pathways, guiding the staff through specific administrative and technical protocols.

- If the staff member requires immediate operational guidance, they can select the **"Chat"** pathway to interact with the integrated, floating LesBot Natural Language Processing (NLP) engine, which provides automated troubleshooting and protocol assistance.
- For primary field operations, staff access the **"Assigned Reports"** module. Here, technicians review pending maintenance tasks, conduct the necessary physical repairs or inspections, and subsequently update the system status. This pathway features a critical ternary decision node regarding the ticket's resolution: if a ticket is marked as "Resolve" or "Rejected," the system automatically triggers a background process to archive the record, preserving the facility's historical data integrity. Conversely, if the ticket is marked as "On Hold," it bypasses the archival process and remains active in the queue.
- To manage financial and disciplinary workflows, staff can navigate to the **"Manage Penalties"** option, enabling them to securely review and confirm student settlement receipts.
- For administrative oversight, the **"User Population"** pathway allows staff to seamlessly count and view the total active user metrics (encompassing administrators, staff, and students).
- Finally, the **"Archive"** pathway grants staff access to the system's audit logs, providing a transparent view of historical operations and resolved tickets.

Following the execution of any selected function, the workflow reaches a deliberate session-control decision point: "Back to staff hub." This structural loop allows personnel to either return to the main dashboard to execute multiple consecutive tasks or safely exit the system by selecting "no," which terminates the session and reaches the "End" state. This highly structured, multi-threaded flowchart ensures that staff can efficiently execute their roles in user management, facility maintenance tracking, and penalty administration, ultimately supporting the continuous and effective operation of the UTeM dormitory ecosystem.

3.3.3. Flowchart for Lestari Dormitory System for Admin

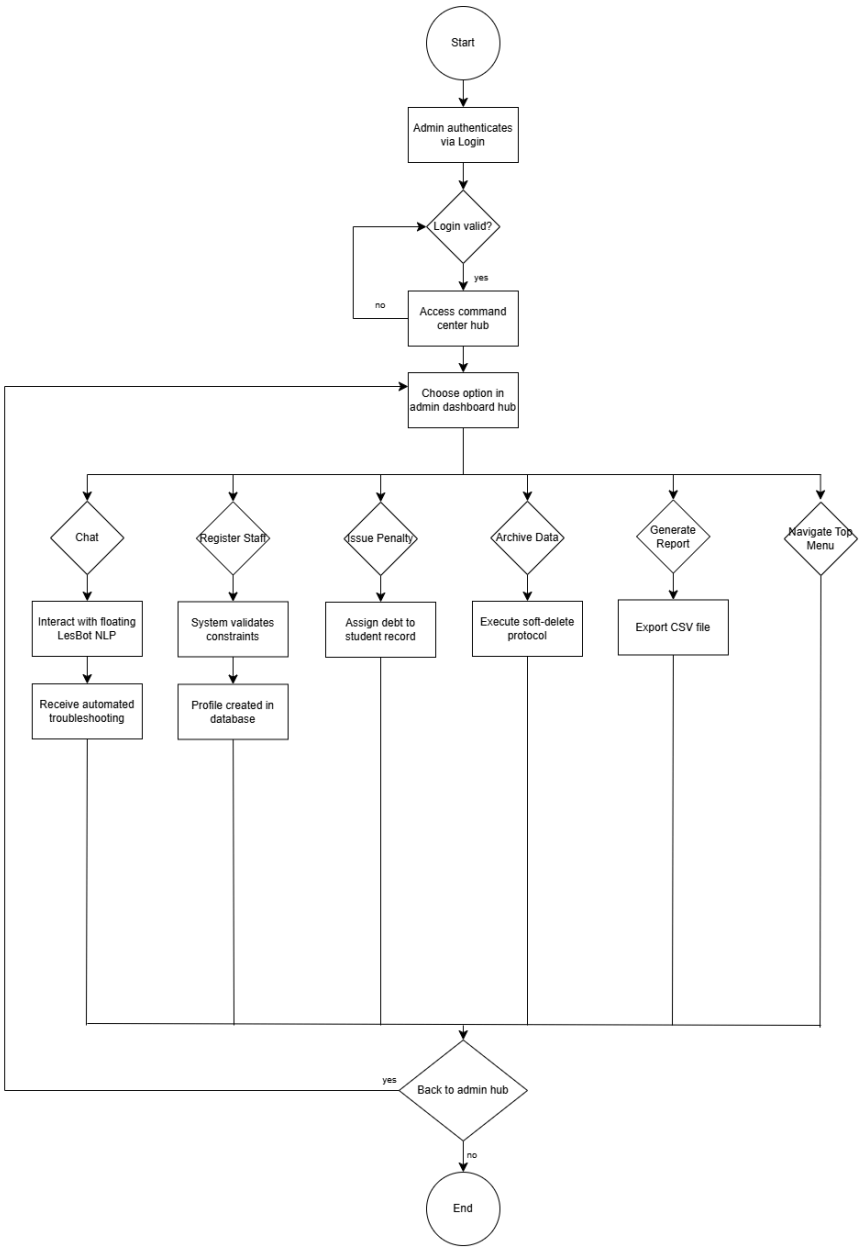


Figure 3.3.3.: Flowchart of Lestari Dormitory System for Admin

Figure 2.3.3.3 illustrates the comprehensive operational workflow for the Administrator module within the Lestari Dormitory System (LesBot). The procedural sequence initiates with a high-level security gateway requiring enterprise-grade credential authentication. A rigorous validation loop protects the system architecture; invalid access attempts continuously redirect the user to the login interface to prevent unauthorized entry. Upon successful verification, the administrator is granted access to the "COMMAND CENTER," which functions as the primary neural hub for overarching system management and facility oversight.

From this centralized dashboard, the system architecture bifurcates into six strategic pathways, encompassing both active operational triggers and systemic monitoring:

- **Chat:** The administrator can engage the floating LesBot NLP interface to receive automated troubleshooting and system protocol assistance.
- **Register Staff:** For user management, the admin initializes new technical personnel profiles. This pathway critically includes a backend systemic check where the database validates entity constraints (such as preventing duplicate identifiers) prior to officially committing the profile to the database.
- **Issue Penalty:** This disciplinary module empowers the administrator to assign financial debt directly to specific student records, ensuring strict adherence to dormitory regulations.
- **Archive Data:** Serving as a vital data management tool, this pathway executes a "soft-delete" protocol. This ensures that obsolete data is securely archived rather than permanently erased, thereby maintaining the relational integrity of the historical ledger.
- **Generate Report:** An analytics function that processes operational metrics and outputs the data via a CSV file export, facilitating external auditing and reporting.
- **Navigate Top Menu:** A systemic oversight pathway that allows the administrator to access read-only enterprise modules—such as Accounts, Maintenance, Penalties, Audit, and Trends—without executing master triggers.

Following the execution of any designated function, the workflow progresses to a deliberate session-control node evaluating whether the user will loop "Back to admin hub." Selecting "yes" seamlessly returns the administrator to the primary dashboard for continuous task execution, while selecting "no" securely terminates the session, bringing the systemic process to an "End." This highly structured flowchart emphasizes advanced database administration principles, secure data handling, and comprehensive facility oversight.

3.4 Requirement analysis of the to-be-system

3.4.1 Functional Requirement (Process Model)

The functional requirements define the specific behaviors of the system. For a database-centric project, these must be mapped to CRUD operations:

- User Authentication: The system must verify credentials against the tbl_accounts table.
- Chatbot Processing: The bot must be able to parse string inputs, search the kb_faq table for matches, and return the appropriate Response_ID.
- Maintenance Ticketing: The system must allow students to INSERT reports and technicians to UPDATE status from 'Pending' to 'Resolved'.
- Penalty Tracking: Administrators must be able to CREATE and DELETE penalty records based on disciplinary actions.

3.4.1.1 Data Flow Diagram Level 0 context diagram for Lestari Dormitory System

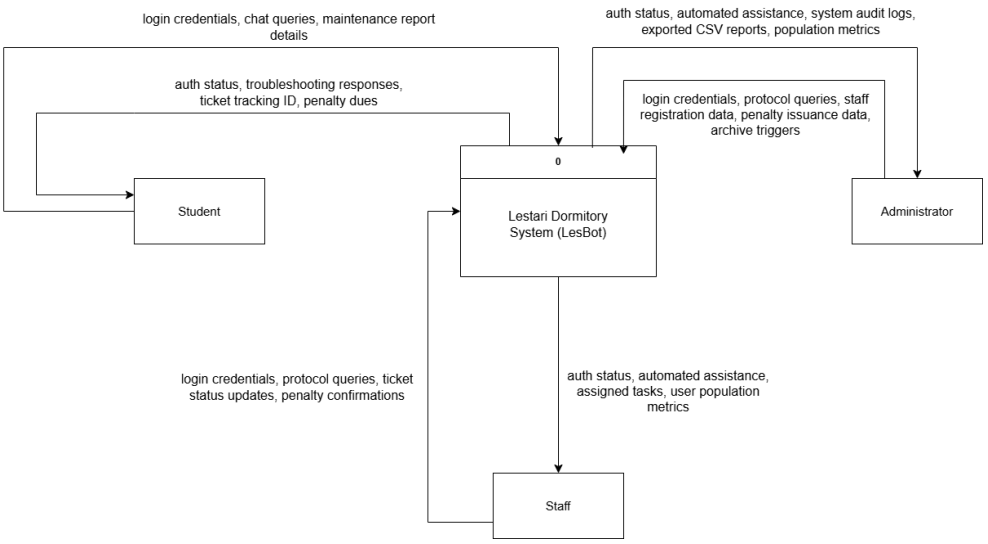


Figure 3.4.1.1 Data Flow Diagram Level 0 context diagram for Lestari Dormitory System

Figure 3..4.1.1 illustrates the context diagram (Level 0 data flow diagram) for the Lestari Dormitory System (LesBot). This macro-level architectural model establishes the absolute boundaries of the proposed digital ecosystem and identifies the three primary external entities that interact with the central processing engine: the student, the staff (technician), and the administrator. By treating the entire LesBot platform as a single, centralized process (Process 0), the diagram maps the high-level exchange of information required to replace the fragmented, manual reporting mechanisms of the previous system.

Acting as the centralized operational hub, the system receives specific data inputs from each entity and returns processed, automated outputs:

- The Student Entity: Initiates interaction by inputting login credentials, chat queries, and maintenance report details into the system. In return, the system processes these requests and outputs the user's auth status, immediate troubleshooting responses via the NLP engine, a generated ticket tracking ID for active reports, and any outstanding penalty dues.
- The Staff Entity: Interfaces with the system to manage field operations by submitting login credentials, protocol queries, ticket status updates, and penalty confirmations. The system responds by delivering the staff's auth status, automated assistance, assigned tasks for maintenance routing, and user population metrics.
- The Administrator Entity: Exercises enterprise oversight by inputting login credentials, protocol queries, staff registration data, penalty issuance data, and archive triggers. The central system subsequently outputs the admin's auth status, automated assistance, forensic system audit logs, exported CSV reports, and global population metrics.

This top-level data mapping successfully demonstrates how LesBot consolidates all residential management into a single, cohesive digital pipeline, ensuring that critical facility data is continuously synchronized and securely processed between all stakeholders.

3.4.1.2. Data Flow Diagram Level 1 for Lestari Dormitory System

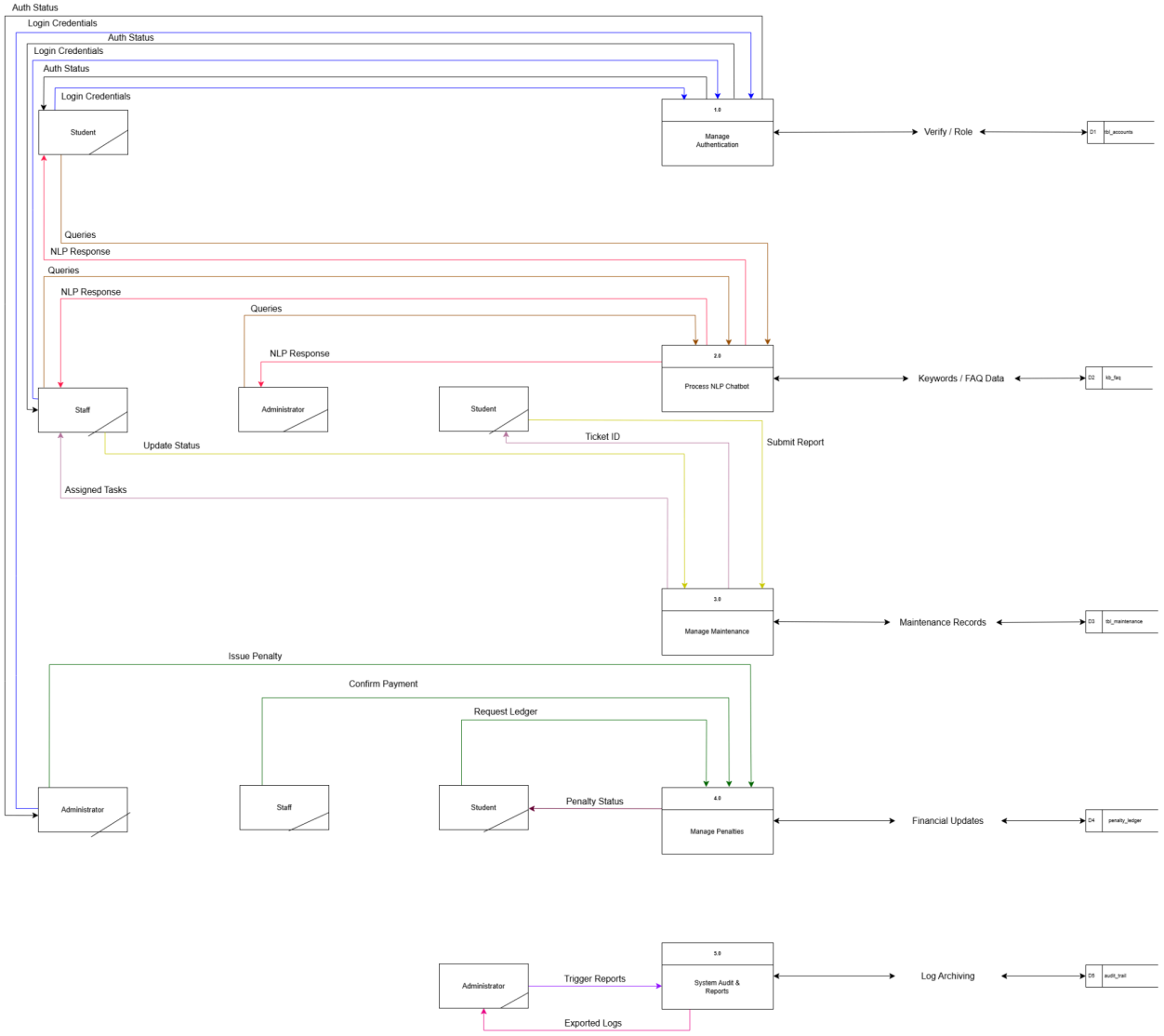


Figure 3.4.1.2. Data Flow Diagram Level 1 for Lestari Dormitory System

Figure 3.4.1.2 provides a detailed functional decomposition of the Lestari Dormitory System (LesBot), moving beyond the context of the overall system to illustrate the internal processing logic. This Level 1 Data Flow Diagram (DFD) highlights the critical interaction between the five primary functional modules and the underlying MySQL data stores, providing a clear map of how data is transformed, retrieved, and stored during typical dormitory operations.

The DFD is structured into five distinct, specialized processes that reflect the system's modular architecture:

1. **1.0 Manage Authentication:** This process acts as the primary security gateway. It accepts login credentials from the Student, Staff, and Administrator entities and performs a verification check against the `tbl_accounts` data store to enforce strict Role-Based Access Control (RBAC).
2. **2.0 Process NLP Chatbot:** This module manages the intelligent troubleshooting component. It processes user chat queries and interacts with the `kb_faq` data store to retrieve and deliver context-aware, automated support, ensuring 24/7 responsiveness.
3. **3.0 Manage Maintenance:** The core operational engine of the system. It facilitates the submission of new reports by students and the updating of ticket statuses by staff. It maintains a persistent state of the facility environment via the `tbl_maintenance` data store, ensuring that administrators can track maintenance workflows in real-time.
4. **4.0 Manage Penalties:** This financial and disciplinary subsystem manages the `penalty_ledger`. It allows administrators to issue penalties, provides students with a transparent view of their current financial status, and enables staff to verify settlement receipts, ensuring accurate record-keeping.
5. **5.0 System Audit & Reports:** This oversight process centralizes administrative reporting. It interacts with the `audit_trail` data store to log all critical system actions—such as staff registration or archive triggers—ensuring transparency and providing the capability to export operational data as CSV reports.

By compartmentalizing these processes and explicitly mapping their interaction with the relational data stores, this DFD demonstrates how LesBot successfully eliminates the data fragmentation issues of the previous manual system. Every data movement is routed through a controlled process circle, ensuring that all information is normalized, securely stored, and readily available for administrative decision-making, thereby supporting the operational excellence of the UTeM residential ecosystem.

3.4.2 Non-Functional Requirement

Non-functional requirements define the quality attributes, constraints, and operational standards of the Lestari Dormitory System. These ensure that the system is not only functional but also secure, reliable, and user-friendly.

1) Security (Data Integrity and Confidentiality)

As a database-centric project, securing sensitive student and administrative data is a primary priority.

Authentication Security: All user passwords must be encrypted using the password_hash() function with the BCRYPT algorithm before being stored in tbl_accounts.
Injection Prevention: To neutralize SQL injection threats, all database interactions must be implemented via PHP Data Objects (PDO) utilizing parameterized prepared statements.
Authorization: The system must enforce Role-Based Access Control (RBAC) to ensure that students, staff, and administrators can only access data pertinent to their specific permissions.

2) **Usability (Accessibility and Responsiveness)**

The system must provide a frictionless experience for a diverse user base.

Responsive Design: Utilizing the Bootstrap 5 framework, the interface must adhere to a "Mobile-First" philosophy. This ensures full functionality across various devices, including smartphones, tablets, and desktop computers.

User Intuition: The NLP chatbot must provide clear, human-readable troubleshooting guidance to reduce the learning curve for new residents.

3) **Availability and Reliability**

Since maintenance issues (e.g., pipe bursts) can occur at any time, the system must remain operational outside of standard office hours.

24/7 Accessibility: The platform, hosted on Azure Cloud Services, must maintain high uptime to support 24/7 maintenance reporting and penalty queries.

Concurrency: The MySQL database must be configured to handle concurrent connections, ensuring system stability during peak usage periods (e.g., end-of-semester penalty settlements).

4) **Performance (Efficiency)**

The system must be optimized for speed to prevent user frustration.

Query Latency: Database lookups for the NLP Chatbot (searching kb_faq) must utilize indexed columns to ensure that response times remain under 200ms.

Page Load Speed: Front-end assets must be optimized to ensure that the user dashboard loads within 2 seconds under standard campus Wi-Fi conditions.

5) **Maintainability**

The system architecture must allow for future upgrades.

Modular Code: The PHP backend must follow a modular structure, allowing developers to update the NLP logic or database schema (e.g., moving from keyword matching to Machine Learning) without re-engineering the entire platform.

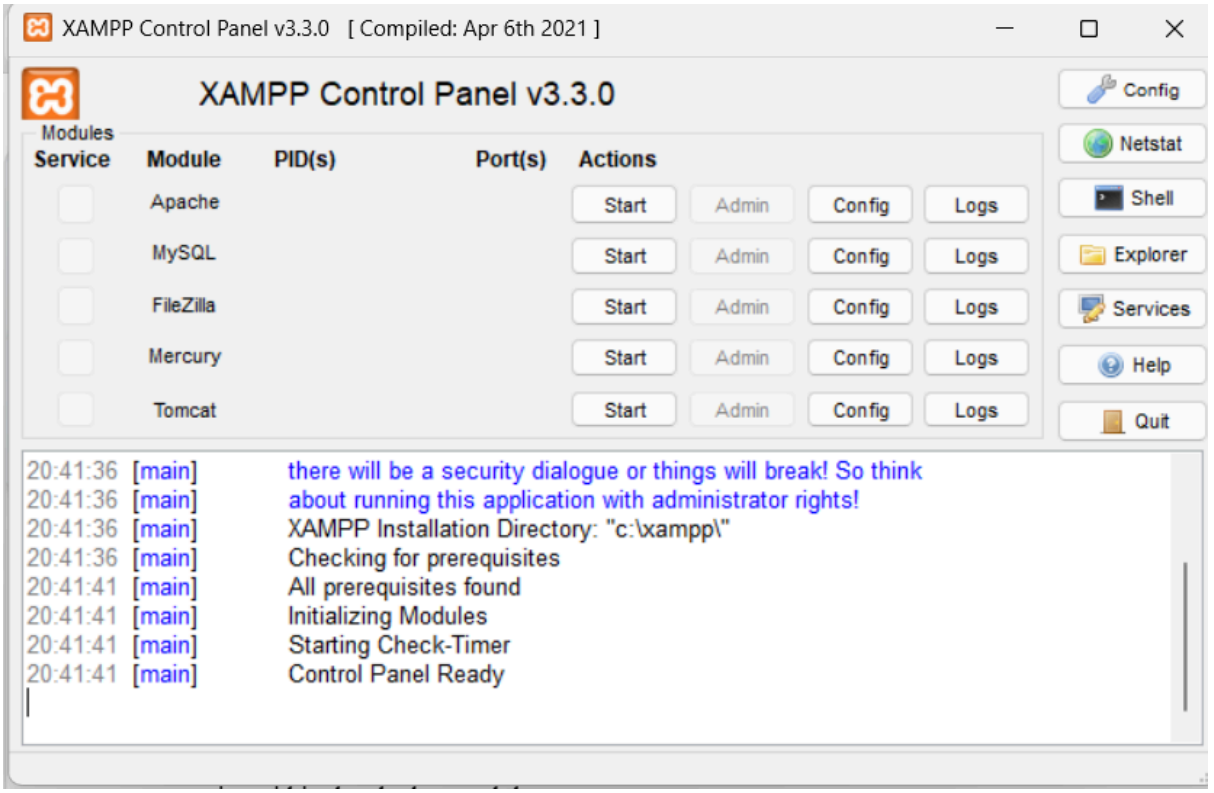
3.4.3 Others Requirement

The implementation and testing of the Lestari Dormitory System require a specific set of software and hardware tools to ensure the development environment mirrors the target production environment (Azure Cloud).

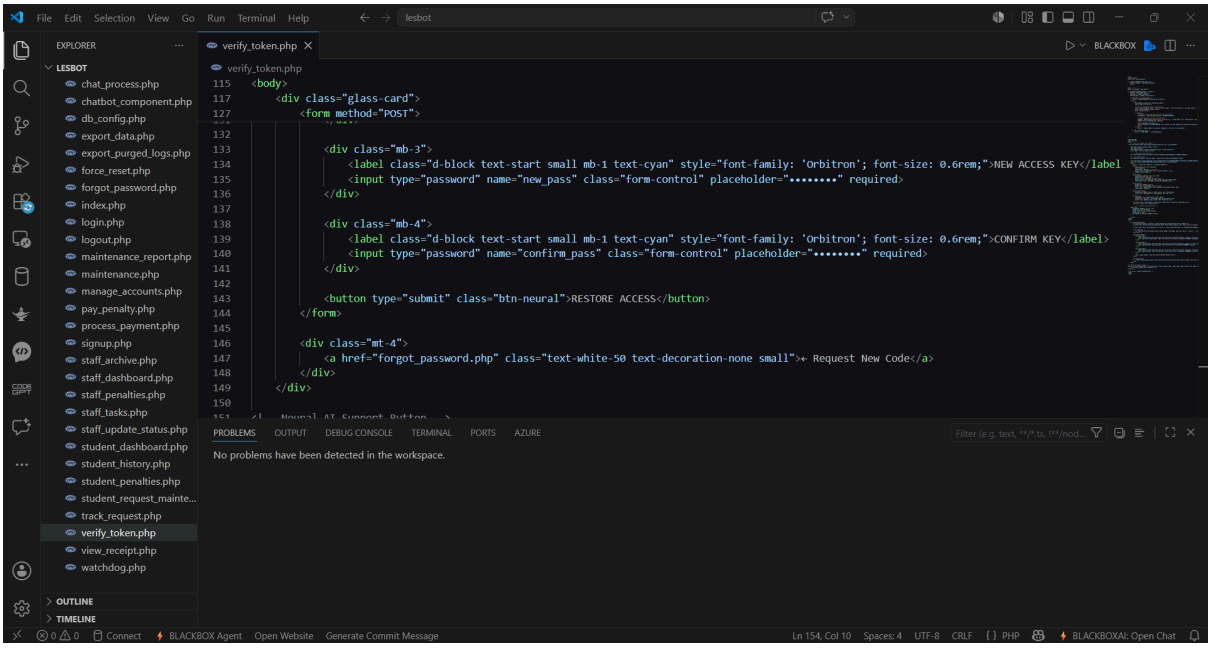
1. **Software Requirements**

The software stack is selected based on compatibility with the XAMPP (AMP) architecture and the requirements for high-performance database management.

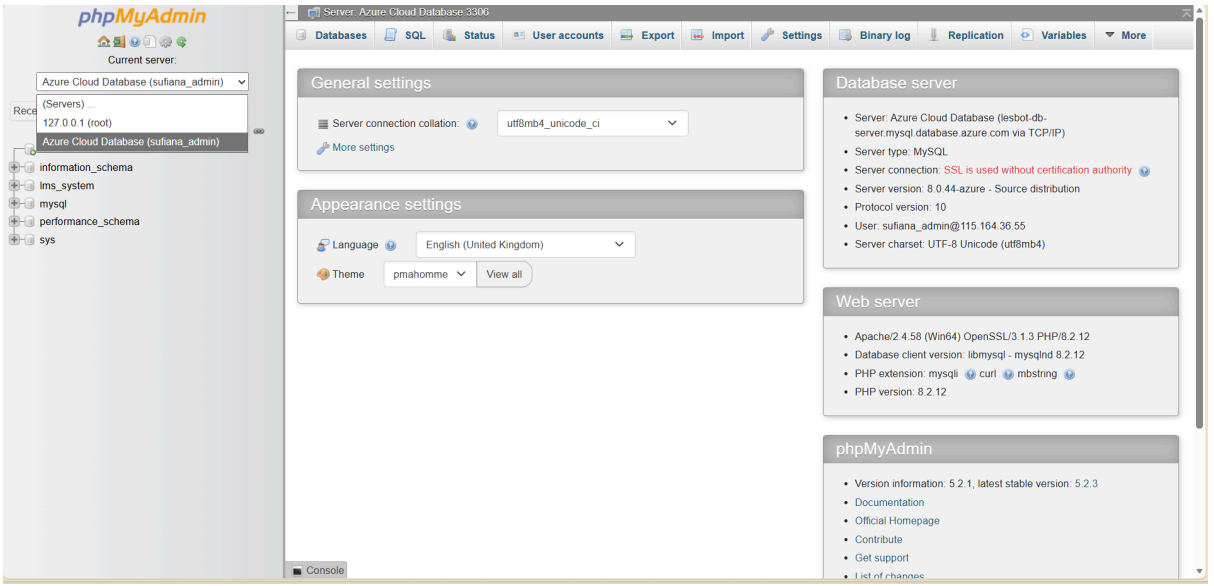
- **Server Stack (Local Development):** XAMPP v8.2.x (encompassing Apache 2.4 web server and MySQL 8.0/MariaDB). This provides the PHP interpreter required for backend logic.



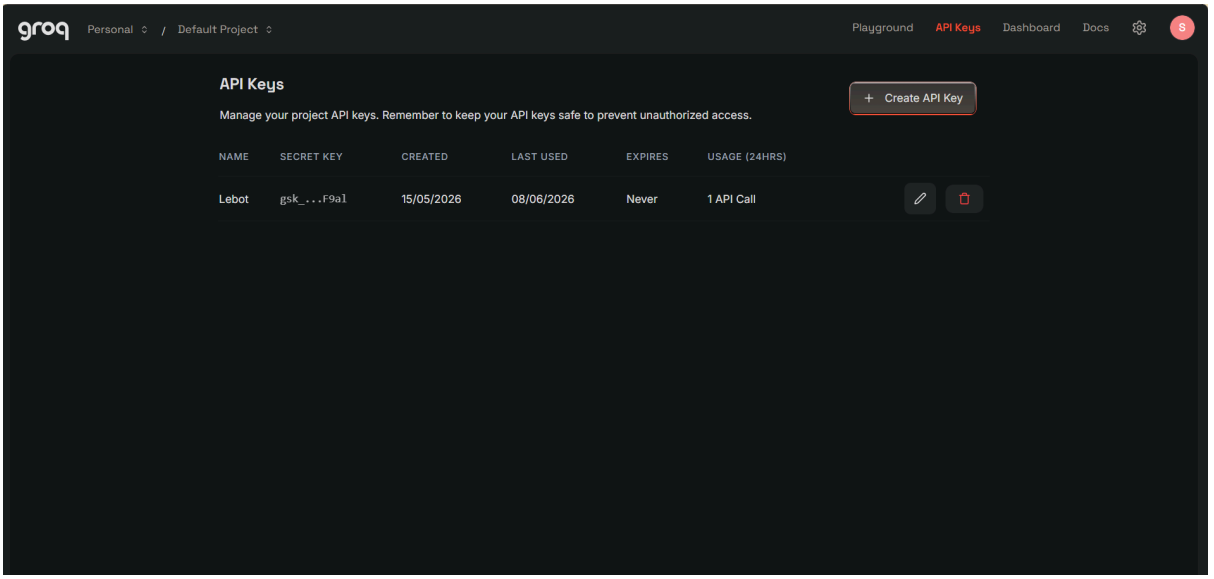
- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) with extensions for PHP IntelliSense and SQL syntax highlighting.



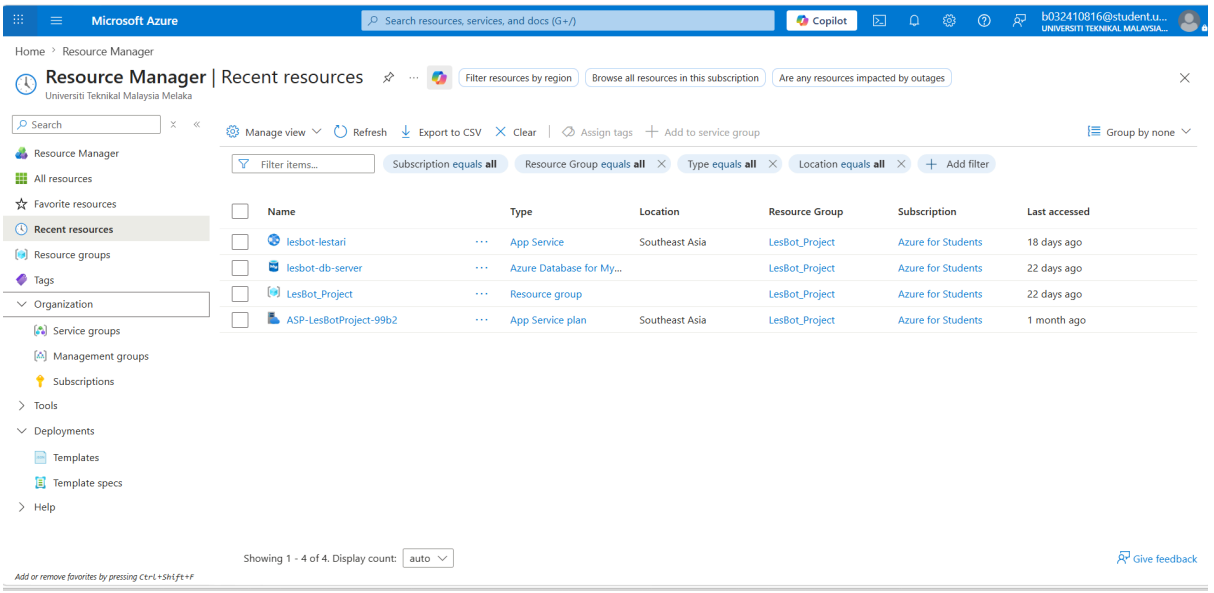
- **Database Administration:** phpMyAdmin 5.2 for local schema prototyping and SQL query optimization.



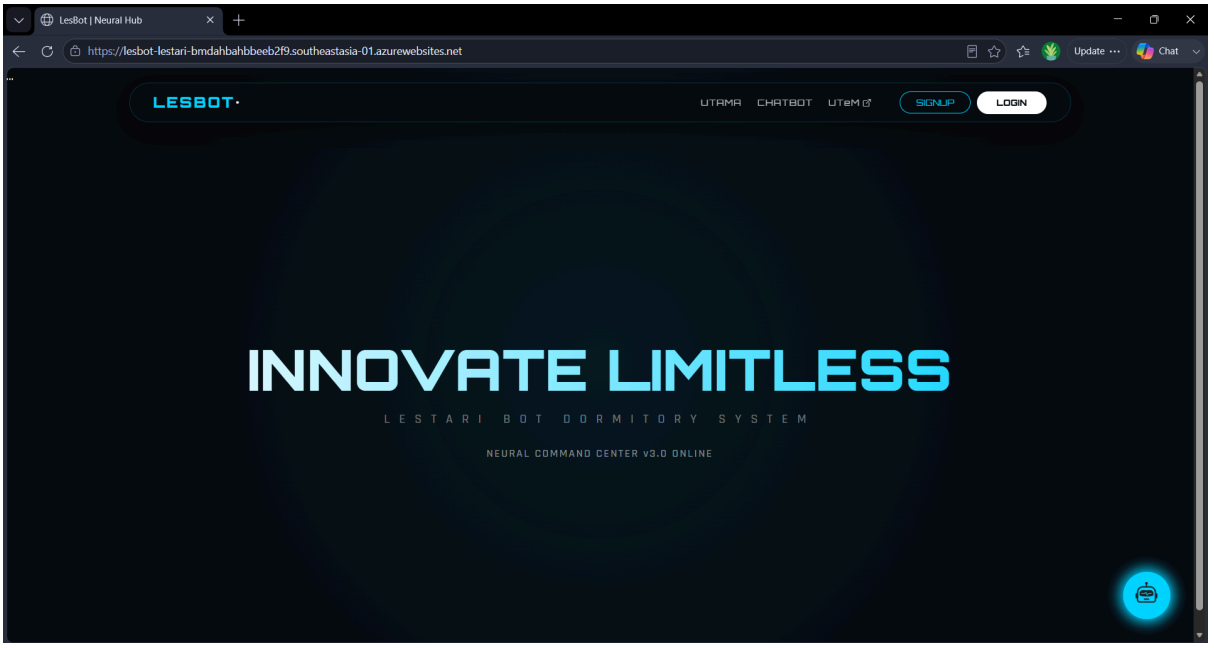
- **External AI Integration:** Google Gemini 1.5 Flash API for processing Natural Language queries within the chatbot module.



- **Cloud Deployment Platform:** Microsoft Azure App Service (Southeast Asia region) for live hosting and global accessibility.



- **Testing & Client Tools:** Modern web browsers (Google Chrome v115+, Microsoft Edge) and Postman for API endpoint validation.



2. Hardware Requirements

The hardware must support the concurrent execution of a web server, a database engine, and heavy AI-API processing during the development lifecycle.

a) Development Station:

- Processor: Intel Core i5 (11th Gen) or equivalent.
- Memory: Minimum 8GB DDR4 RAM to prevent bottlenecking during concurrent local server and IDE usage.
- Storage: 256GB SSD for rapid data indexing and read/write operations.

b) Client-Side Testing Devices:

- Mobile Device: Standard Android/iOS smartphone with a minimum resolution of 1080p to validate the Bootstrap 5 responsive grid system.
- Desktop PC: For testing the Administrative Dashboard and large-scale CSV report generation.

3.5 Conclusion

Chapter 3 has provided a comprehensive analytical view of why Lestari-Bot is a necessary evolution for dormitory management. Through the PIECES Framework, it was demonstrated that the current system suffers from significant information fragmentation and service delays. By defining the functional requirements of the chatbot and the non-functional security needs of the database, this analysis sets the stage for the technical design. The findings here lead directly into Chapter 4: Design, where these requirements are converted into a normalized database schema and user interface storyboards. The definition of Functional Requirements through CRUD operations and the mapping of system logic via the Level 1 Data Flow Diagram (DFD) ensure that the "To-Be" system is built on a

logical, balanced architecture. Furthermore, the Non-Functional Requirements prioritize data security and system availability, which are essential for an institutional-grade management platform.

CHAPTER 4. DESIGN

4.1 Introduction

The design phase translates the analytical requirements established in Chapter 3 into a technical blueprint. For Lestari-Bot, this involves a dual focus: the structural integrity of the back-end database and the intuitive flow of the front-end user interface. This chapter details the architectural transition of the dormitory system, moving from a basic C++ data structure to a professional-grade relational database. By adhering to rigorous design principles, the project ensures that the system is not only functional but also scalable and secure against data anomalies and the design phase serves as the technical translation of the analytical requirements established in Chapter 3. For the Lestari Dormitory System (LesBot), this phase encompasses the transition from abstract system logic to a concrete technical blueprint. The design follows a dual-track approach: ensuring the structural integrity of the normalized MySQL backend while optimizing the user experience (UX) through a responsive front-end interface. By adhering to formal software engineering principles, the project ensures a system that is not only functional but also scalable and resistant to data anomalies.

4.2 Introductory preview to this chapter

This chapter is divided into two primary segments. The first segment, Database Design, explores the evolution of the data model through Conceptual, Logical, and Physical stages, with a heavy emphasis on 3rd Normal Form (3NF) normalization. The second segment, Graphical User Interface (GUI) Design, showcases the storyboards and wireframes for the web portal and the chatbot session, ensuring that the presentation layer effectively communicates with the data layer.

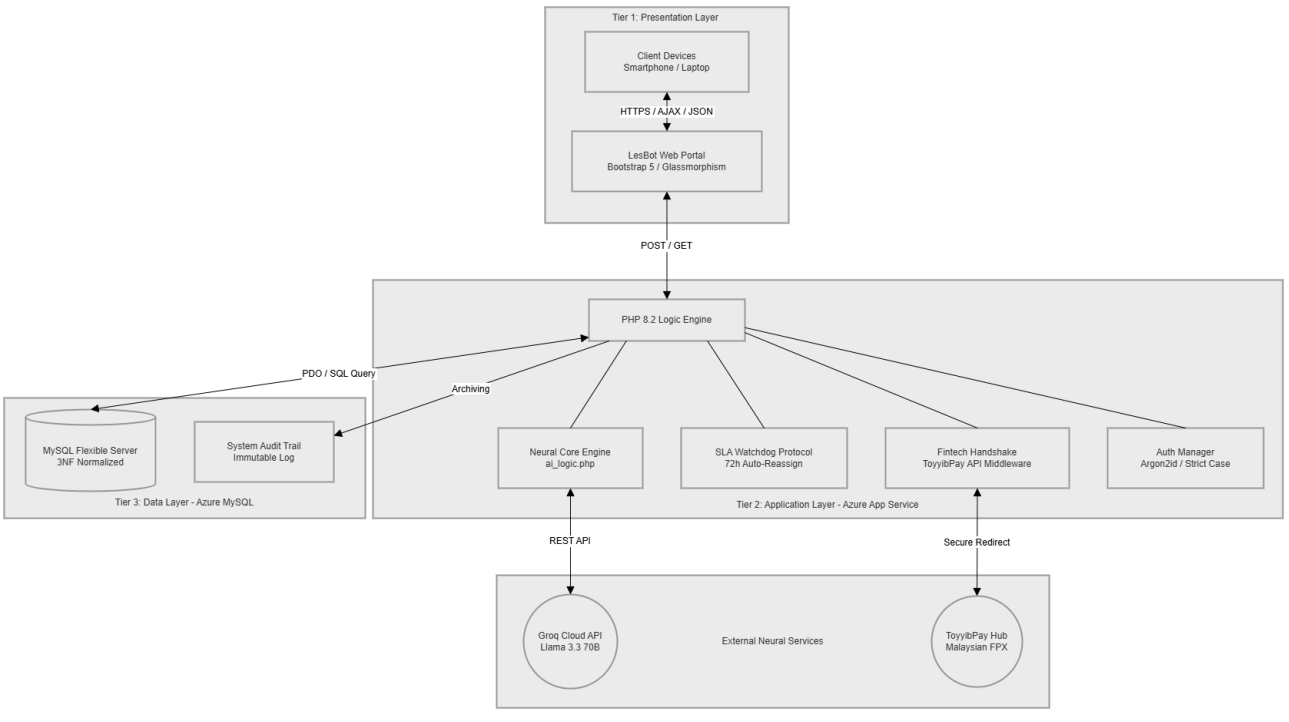


Figure 4.2. 3-Tier Architecture for Lestari Dormitory System

Tier 1: Presentation Layer (Client-Side)

This is the "Top" layer. It represents what the user sees and interacts with.

- **Components to draw:**

1) Devices: Icons for a Smartphone (Responsive UI) and a Laptop.
2) Web Interface: Label this as "LesBot Web Portal".
3) Technologies: HTML5, CSS3 (Glassmorphic design), JavaScript (AJAX), Bootstrap 5.

- **User Roles:**

1) Student (Dashboard & Chatbot)
2) Staff (Field Ops Modal)
3) Admin (Strategic BI Trends)

- **Interaction:** Users send HTTPS Requests and receive JSON/HTML Responses.

Tier 2: Application Layer (Logic/Server-Side)

This is the "Middle" layer. It is the "Brain" hosted on the cloud.

- **Primary Server: Microsoft Azure App Service** (Linux/PHP 8.2 Environment).

- **Core Logic Modules (Draw these inside the server box):**

1) Auth Module: High-level Argon2id encryption logic.
2) Neural Core: The ai_logic.php script that manages AI sessions.
3) SLA Watchdog: The background script that reassigns "On-Hold" tickets after 72 hours.
4) Fintech Module: Handshake logic for FPX payments.

- **External API Integration (Draw these branching out to the side):**

1) Groq Cloud API: Link this to the Neural Core (Model: Llama 3.3 70B).
2) ToyyibPay API: Link this to the Fintech Module (Real-world FPX Settlement).

Tier 3: Data Layer (Persistence Layer)

This is the "Bottom" layer. It ensures every action is recorded and can never be changed.

- **Database Server: Azure Database for MySQL (Flexible Server).**

- **Key Tables (List these inside the DB box):**

1) USERS (Identity Provider)
2) STUDENT / STAFF (Role Profiles)

3) MAINTENANCE_REQUEST (Complaint Records)
4) SYSTEM_AUDIT_TRAIL (Immutable Logs)
5) TBL_CHAT_LOG (Neural Memory)

- **Security Context:** Show a Firewall icon between Tier 2 and Tier 3, representing the Azure Client IP Whitelisting we configured.

4.3 Database Design

As the core component of a BITD project, the database design for Lestari-Bot is engineered to handle complex relationships between students, administrative staff, and automated chatbot logs.

4.3.1 Conceptual Design

The conceptual design is represented by the Entity Relationship Diagram (ERD). This high-level model identifies the primary entities—Student, Staff, Maintenance_Ticket, Penalty, and Chat_Session—and defines their cardinalities.

- **One-to-Many Relationships:** A single student can initiate multiple chatbot sessions and file multiple maintenance tickets.
- **Many-to-Many Relationships**: Multiple staff members may be assigned to various maintenance tickets over time, resolved through an associative entity to maintain historical accuracy.

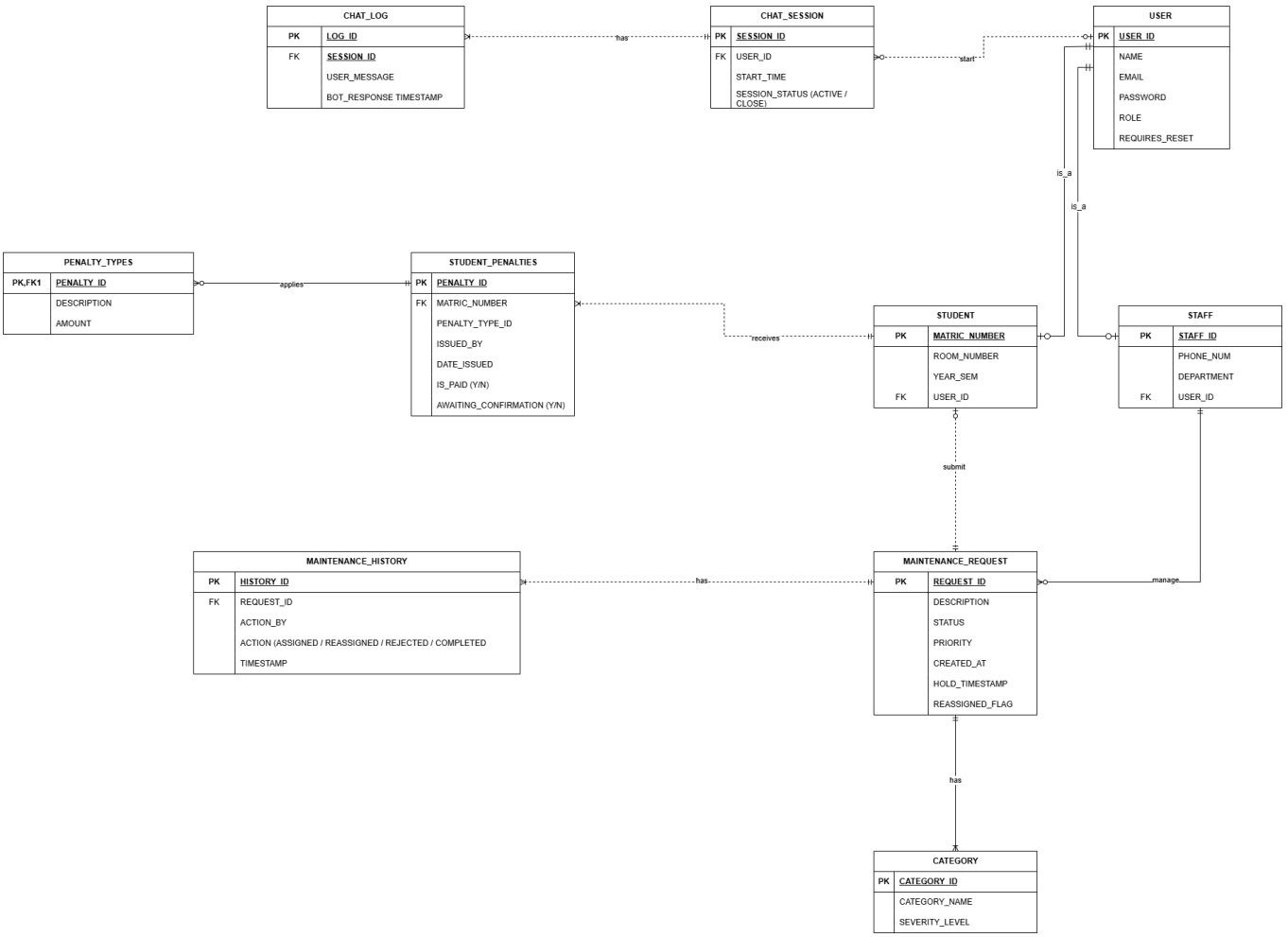


Figure 4.3.1. ERD for Lestari Dormitory System

Business Rules for Lestari Dormitory System:

a) A User may be associated with zero or one Student record.

b) Each Student record must be associated with exactly one User.

c) A User may be associated with zero or one Staff record.

d) Each Staff record must be associated with exactly one User.

e) A Student may submit zero or many Maintenance Requests.

f) Each Maintenance Request must be submitted by exactly one Student.

g) A Category may classify zero or many Maintenance Requests.

h) Each Maintenance Request must be classified by exactly one Category.

i) A Staff member may be assigned to zero or many Maintenance Requests.

j) Each Maintenance Request may be assigned to zero or one Staff member.

k) A Maintenance Request may have zero or many Maintenance History entries.

l) Each Maintenance History entry must belong to exactly one Maintenance Request.

m) A Student may incur zero or many Student Penalties.

n) Each Student Penalty must be incurred by exactly one Student.

o) A Penalty Type may define zero or many Student Penalties.

p) Each Student Penalty must be defined by exactly one Penalty Type.

q) A Student Penalty may generate zero or one Student Payment (Receipt).

r) Each Student Payment must settle exactly one Student Penalty.

s) A User may initiate zero or many Chat Sessions with LesBot.

t) Each Chat Session must be initiated by exactly one User.

u) A Chat Session may contain zero or many Chat Logs (Messages).

v) Each Chat Log must belong to exactly one Chat Session.

w) An Admin (User) may perform zero or many actions recorded in the System Audit Trail.

x) Each entry in the System Audit Trail must be logged by exactly one Admin.

4.3.2 Logical Design

The logical design focuses on the Normalization Process, which is the hallmark of a Database degree project. This ensures that the Lestari-Bot system is free from insertion, update, and deletion anomalies.

- **1st Normal Form (1NF):** LesBot ensures all attributes are atomic. Multi-valued attributes, such as staff_phone, are decomposed into individual records. Every row is uniquely identified by a Primary Key, and there are no repeating groups in the tbl_chat_log or student_payments tables.
- **2nd Normal Form (2NF):** All non-key attributes are fully functionally dependent on the primary key. In the initial Workshop 1 draft, student room details were partially dependent on the maintenance ticket ID. In the current schema, these are separated: room_number depends only on matric_number in the student table, removing partial dependencies.
- **3rd Normal Form (3NF):** The system eliminates transitive dependencies. For example, in the student_penalties table, the fine amount and violation description were originally dependent on the penalty_id. These have been moved to a lookup table, penalty_types. This ensures that a change in the "Standard Fine" for smoking only needs to be updated in one place, rather than updating every historical student record.

4.3.2.1 Data Dictionary

4.3.2.1.1. users Table

The central identity anchor for the entire neural network.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
`user_id`	Varchar(30)	Yes	Primary Key	
`name`	Varchar(100)	Yes		
`email`	Varchar(100)	Yes		Unique Index
`password`	Varchar(255)	Yes		Argon2id Hash
`role`	Enum (‘Student, ‘Staff’, ‘Admin’)	Yes		
`requires_reset`	Tinyint	Yes		Default: 0
`reset_token`	Varchar(64)	No		
`token_expiry`	Datetime	No		

4.3.2.1.2. student Table

Stores residency metadata linked 1:1 to the Users table.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
`matric_number`	Varchar(30)	Yes	Primary Key /Foreign Key	`users (user_id)`
`room_number`	Varchar(20)	Yes		RegeX Enforced
`year_sem`	Varchar(30)	Yes		

4.3.2.1.3. staff Table

Stores operational metadata linked 1:1 to the Users table.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
`staff_id`	Varchar(30)	Yes	Primary Key /Foreign Key	`users (user_id)`
`phone_num`	Varchar(20)	No		
`department`	Varchar(50)	Yes		Default: ‘Maintenance’

4.3.2.1.4. category Table

Lookup table for maintenance classification and severity triage.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
category_id	Int	Yes	Primary Key	Auto-Increement
category_name	Varchar(50)	Yes		
severity_level	Enum(50)	Yes		Low to Critical

4.3.2.1.5. maintenance_request Table

Transactional table for all facility management reports.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
request_id	Varchar(20)	Yes	Primary Key	
student_id	Varchar(30)	Yes	Foreign Key	student (matric_number)
category_id	Int	Yes	Foreign Key	category (category_id)
description	Text	Yes		
status	Varchar(20)	Yes		SLA Aware
priority	Varchar(20)	Yes		
assigned_staff_id	Varchar(30)	No	Foreign Key	staff (staff_id)
created_at	Timestamp	Yes		Default: Current_timestamp
hold_timestamp	Datetime	No		72h Watchdog Trigger
rejected_count	Int	Yes		Default: 0 (Max: 3)

4.3.2.1.6. penalty_types Table

Standardized lookup for hostel rule violations.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table

penalty_type_id	Int	Yes	Primary Key	Auto-Increement
description	Varchar(100)	Yes		
default_amount	Decimal(10,2)	Yes		

4.3.2.1.7. student_penalties Table

The financial ledger for student disciplinary records.

Attribute Name	**Data Type & Size**	**Required?**	**PK/FK**	**Reference Table**
penalty_id	Int	Yes	Primary Key	Auto-Increement
matric_number	Varchar(20)	Yes	Foreign Key	student (matric_number)
penalty_type_id	Int	Yes	Foreign Key	penalty_types (id)
amount	Decimal(10,2)	Yes		
date_issued	Datetime	Yes		Default: Current_timestamp
is_paid	Tinyint(1)	Yes		Default: 0
remarks	Text	No		For “Others” category

4.3.2.1.8. payment_transactions Table

The Fintech Vault storing real-world bank handshake data.

Attribute Name	**Data Type & Size**	**Required?**	**PK/FK**	**Reference Table**
transaction_id	Varchar(50)	Yes	Primary Key	Bank Reference
penalty_id	Int	Yes	Foreign Key	student_penalties (id)
amount_penalty	Decimal(10,2)	Yes		
fpx_gateway_fee	Decimal(10,2)	Yes		Default: 1.00
total_paid	Decimal(10,2)	Yes		amount + fee
bank_name	Varchar(50)	Yes		e.g., Maybank, CIMB
status	Varchar(20)	Yes		API Veriified

4.3.2.1.9 system_audit_trail Table

The immutable governance log for administrative accountability.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
audit_id	Int	Yes	Primary Key	Auto-Increement
admin_id	Varchar(30)	Yes	Foreign Key	users (user_id)
action_type	Varchar(50)	Yes		E.g., entity_purge
target_entity	Varchar(50)	No		
action_details	Text	Yes		

4.3.2.1.10 tbl_chat_log Table

Stores the neural conversation data for AI analysis.

Attribute Name	Data Type & Size	Required?	PK/FK	Reference Table
log_id	Int	Yes	Primary Key	Auto-Increement
session_id	Int	Yes	Foreign Key	tbl_chat_session
user_message	Text	Yes		
bot_response	Text	Yes		
timestamp	Timestamp	Yes		Default: Current_timestamp

a. Multi-Table Join Query (Neural Tracking View)
   - Used to generate the "Track Requests" dashboard for students by merging data from three tables.

```
SELECT DISTINCT mr.request_id, c.category_name, mr.status, u.name as staff_name
FROM maintenance_request mr
JOIN category c ON mr.category_id = c.category_id
LEFT JOIN users u ON mr.assigned_staff_id = u.user_id
WHERE mr.student_id = :id;
```

b. Aggregate Query (Staff Performance BI)
   - Used by the Admin to calculate the Efficiency Index (%) of technical staff.

```
SELECT u.name,
```

(SUM(CASE WHEN status = 'Completed' THEN 1 ELSE 0 END) / COUNT(*)) * 100 as efficiency_rate

FROM maintenance_request mr

JOIN users u ON mr.assigned_staff_id = u.user_id

GROUP BY u.name;

c. Logical Logic Constraint (SLA Watchdog)
   - A subquery-based logic used by the system to identify "SLA Breaches" where staff have held a report for too long.

UPDATE maintenance_request

SET assigned_staff_id = NULL, status = 'Pending'

WHERE status = 'On-Hold' AND hold_timestamp <= NOW() - INTERVAL 3 DAY;

4.3.3 Physical Design

The physical design translates the logical model into the MySQL environment. This section includes the Data Dictionary, which serves as the definitive technical reference for the system's tables.

4.3.3.1. Selection of DBMS

MySQL 8.0 Flexible Server (Azure Cloud) was selected as the Database Management System for LesBot due to the following technical justifications:

- Transactional Atomicity (ACID Compliance): Crucial for the Fintech Hub integration (ToyyibPay), ensuring that financial settlements and ledger updates either complete fully or roll back entirely to prevent data corruption.
- Native JSON Support: Efficiently stores neural metadata and intent-detection logs from the Llama 3.3 AI engine.
- Cloud Scalability: Provides "Limitless" performance, allowing the database to handle concurrent connections during peak dormitory periods (e.g., end-of-semester check-outs) without latency.
- Flexible Server Architecture: Allows for vertical scaling of compute resources and precise control over database parameters like only_full_group_by and time_zone.

4.3.3.2. Database Objects: Triggers and Constraints

LesBot utilizes specialized database objects to automate business logic and maintain an immutable audit trail:

- Stored Logic (SLA Watchdog): While traditional systems rely on manual monitoring, LesBot uses a time-triggered reassignment protocol. The system evaluates the hold_timestamp attribute; if the duration exceeds 72 hours, the logic layer triggers an update to reset the assigned_staff_id, ensuring no maintenance request is neglected.
- Foreign Key Constraints: Enforces referential integrity with ON DELETE CASCADE on non-permanent tables (like student_payments) and ON DELETE RESTRICT on core identity tables (users) to prevent accidental deletion of historical data.
- Search Optimization (Indexing): To ensure 24/7 responsiveness, indexes are applied to user_id, email, and request_id. This reduces query execution time for authentication and ticket tracking to under 100ms.

4.3.3.3. Security Mechanisms

As a centralized portal for UTeM residents, data security is implemented at multiple levels:

- Neural Access Control (RBAC): Access to tables is restricted based on the role attribute. The Application Layer utilizes specific database users with limited privileges (e.g., the web user can SELECT from users but cannot DROP tables).
- Cryptographic Hashing: LesBot implements Argon2id—the current industry standard for password security. Unlike SHA-256, Argon2id provides resistance against GPU-based brute-force attacks and side-channel attacks.
- Azure Networking & Firewall: The physical server is shielded by a Client IP Whitelist. Only the Azure App Service IP and the developer's authorized workstation are permitted to establish a "Neural Link" with the MySQL port (3306), neutralizing external SQL injection and unauthorized access attempts.

4.3.3.4. Database Contingency: Backup and Recovery

To ensure the system is truly "Limitless" and resilient to disaster, the following contingency protocols are active:

- Point-in-Time Recovery (PITR): Azure maintains automated snapshots of the LesBot database. In the event of a catastrophic data failure, the DBA can restore the system to any specific second within the last seven days.
- Automated Backups: Daily full backups and frequent transaction log backups are stored in redundant Azure storage, protecting the dormitory's historical ledger and room assignments from hardware failure.
- Export Protocols: The system includes a manual export module (export_data.php) that allows Administrators to generate off-site CSV backups of student and maintenance records for secondary archiving.

4.4 Craphical User Interface (GUI) Design

The LesBot GUI is engineered to provide a high-contrast, "Cyberpunk-Neural" aesthetic that emphasizes clarity and speed. Adhering to the Mobile-First philosophy via the Bootstrap 5 framework, the system ensures that students can report critical room failures via smartphone as easily as an Administrator can manage staff performance on a desktop workstation.

4.4.1 Navigation Flow

The system's architecture utilizes a Hierarchical Navigation Flow to maintain strict session integrity and Role-Based Access Control (RBAC):

- Authentication Gate: The entry point where the user provides strict, case-sensitive credentials. Successful login initiates a PHP session.
- Central Hub (Dashboard): A role-specific landing page. Students see their room status and debt; Staff see active field tasks; Admins see global system health.
- Functional Modules: Users navigate to specific tasks (e.g., Report Issue, Neural Ledger, BI Trends) via a floating "Neural Nav" bar that remains persistent across the session.
- Neural AI Overlay: A 24/7 floating robot icon available on every page, allowing users to trigger a chatbot session for instant guidance without leaving their current task.

4.4.2 Input and Output Analysis

In alignment with the functional requirements in Chapter 3, the following table defines the primary data exchange points within the GUI:

Interface Component	Primary Input (Data Entry)	Primary Output (System Response)
Neural Login	user_id, password (Strict case)	Authorized Session & Redirect
Chatbot Helpflow	Natural Language Strings (Malay/English)	Context-Aware AI Advice (Llama 3.3)
Maintenance Report	category_id, priority, description	Generated request_id & Auto-Assignment
Staff Analysis Modal	status_id (Completed, On-Hold, Rejected)	Immutable Audit Entry & Reassignment
Neural Ledger	payment_method (FPX/TNG Selection)	Secure Bank Hub Redirect & Digital Receipt
Admin BI Trends	Date Range & Search Queries	Visual Pie Charts & Strategic AI Report

4.4.3 Interaction Design and Data Validation

To ensure the integrity of the Data Warehouse, LesBot implements Multi-Layer Validation:

- Real-Time Feedback: Forms use JavaScript to provide instant feedback. For example, the Room Number field enforces the [Block]-[Aras]-[Rumah]-[Bilik] format (e.g., B1-1-C-04) using Regex patterns.
- Visual Hierarchy: High-priority items, such as "Urgent" maintenance or "Unpaid" penalties, are categorized with neon color codes (Red/Amber) to alert the user immediately.
- Asynchronous Communication (AJAX): The chatbot interface utilizes AJAX to communicate with the chat_process.php logic. This allows for a "Limitless" conversational feel where the page does not need to refresh to receive AI responses.

4.4.4 User Interface Design

4.4.4.1 Main Page Interface

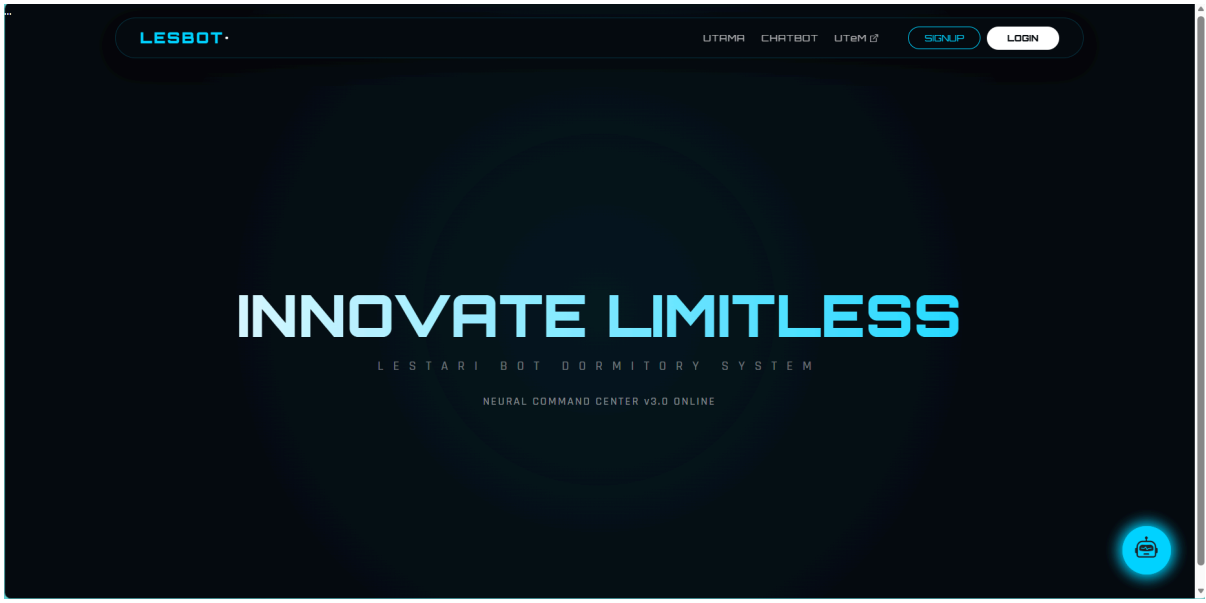


Figure 4.4.4.1 Main Page Interface

4.4.4.2 Login Interface

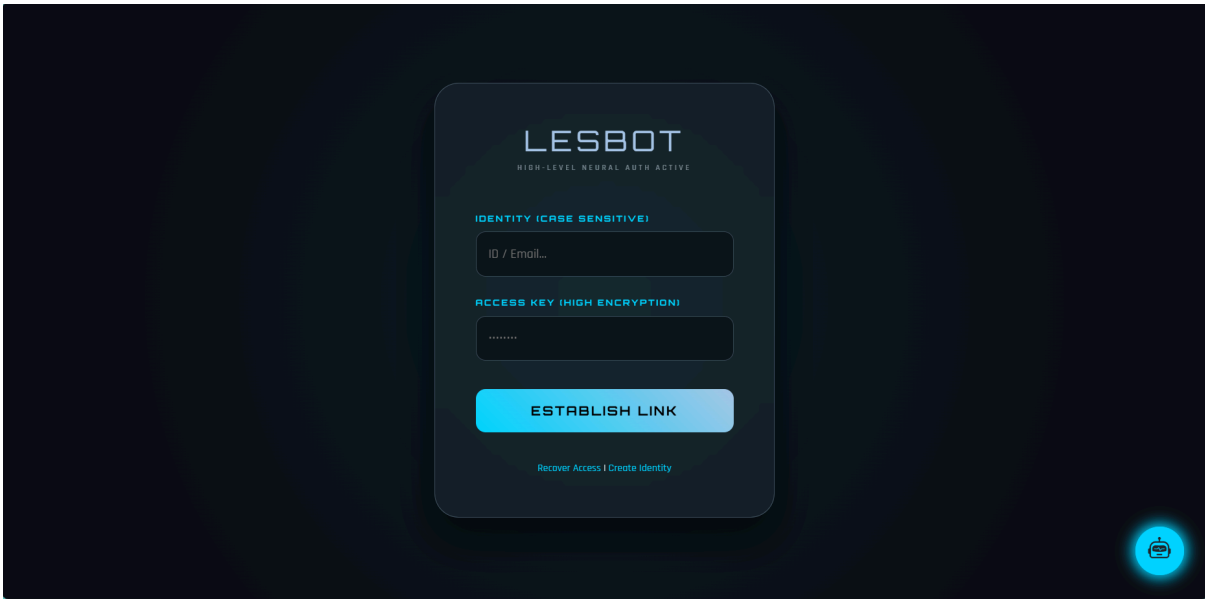


Figure 4.4.4.2 Login Interface

4.4.4.3 Student Dashboard Interface

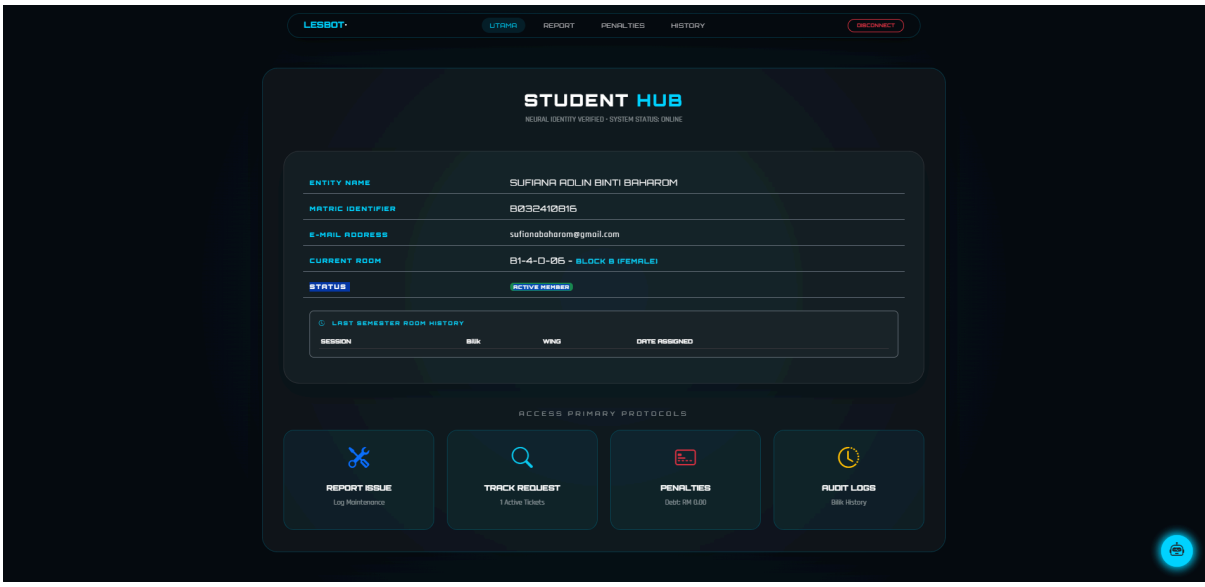


Figure 4.4.4.3 Student Dashboard Interface

4.4.4.4 Student Report Issues Interface

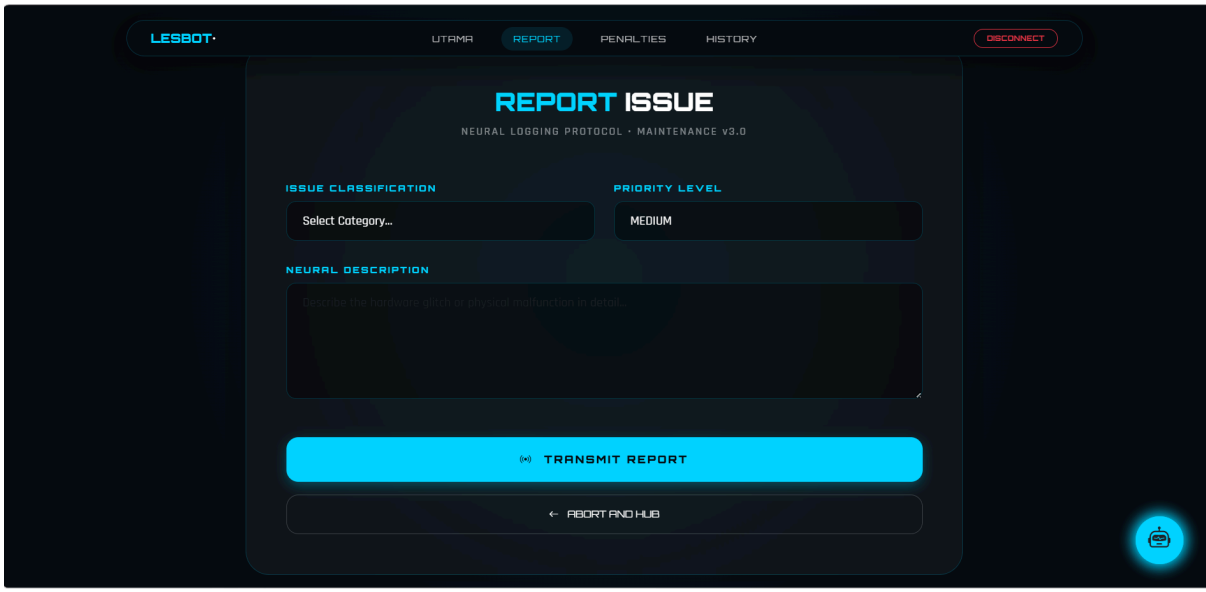


Figure 4.4.4.4 Student Report Issues Interface

4.4.4.5 Student Track Maintenance Request Interface

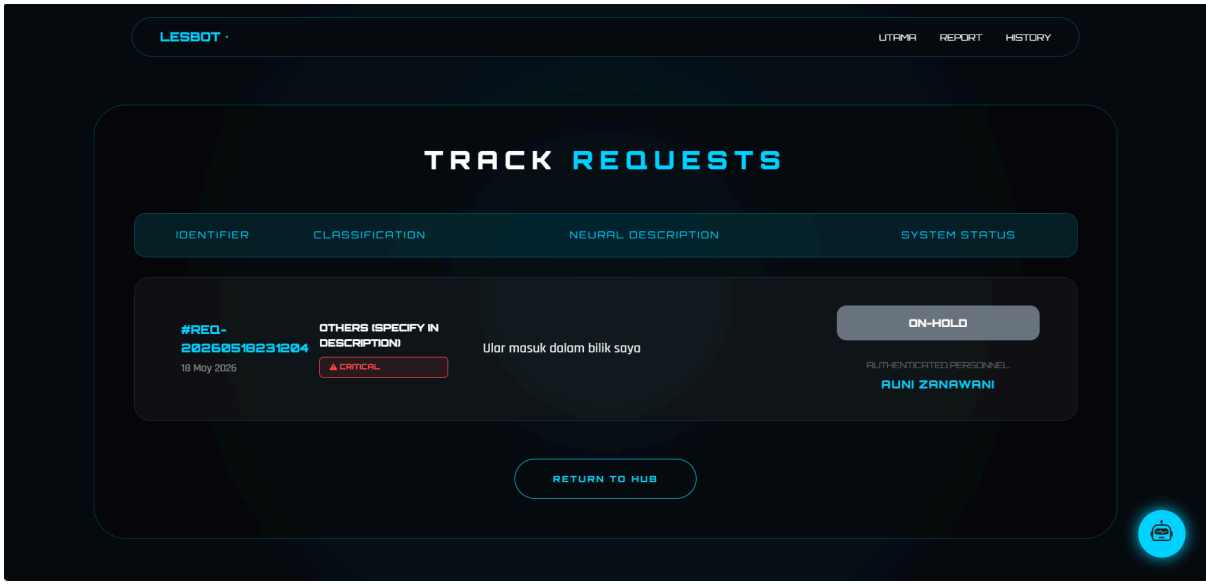


Figure 4.4.4.5 Student Track Maintenance Request Interface

4.4.4.6 Student Penalties Interface

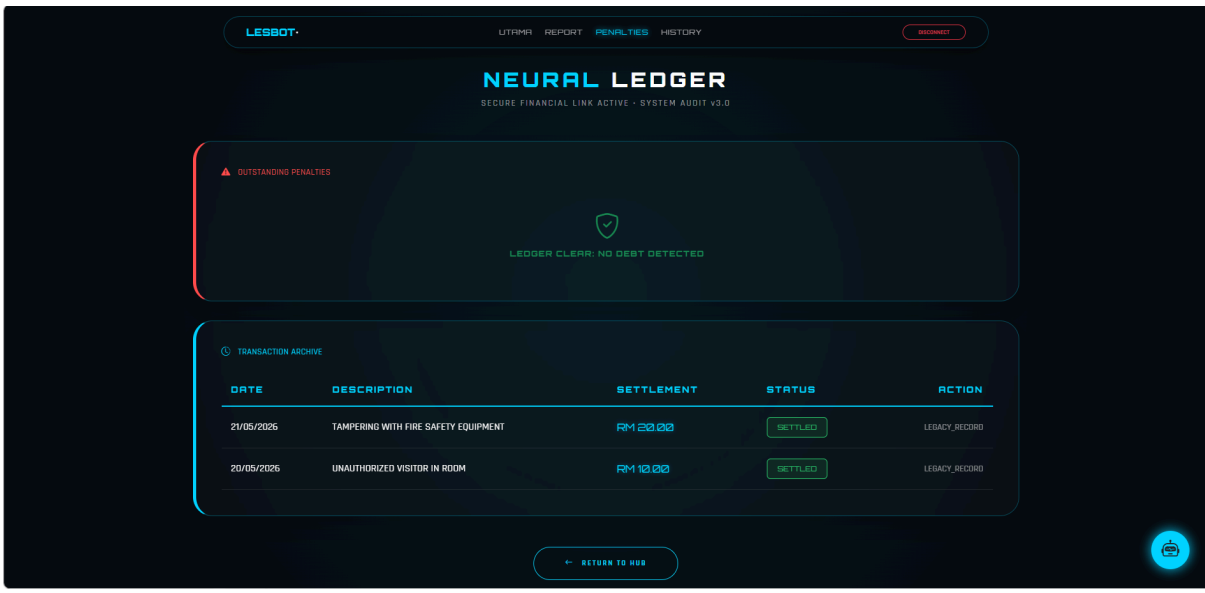


Figure 4.4.4.6 Student Penalties Interface

4.4.4.7 Student Audit Logs Interface

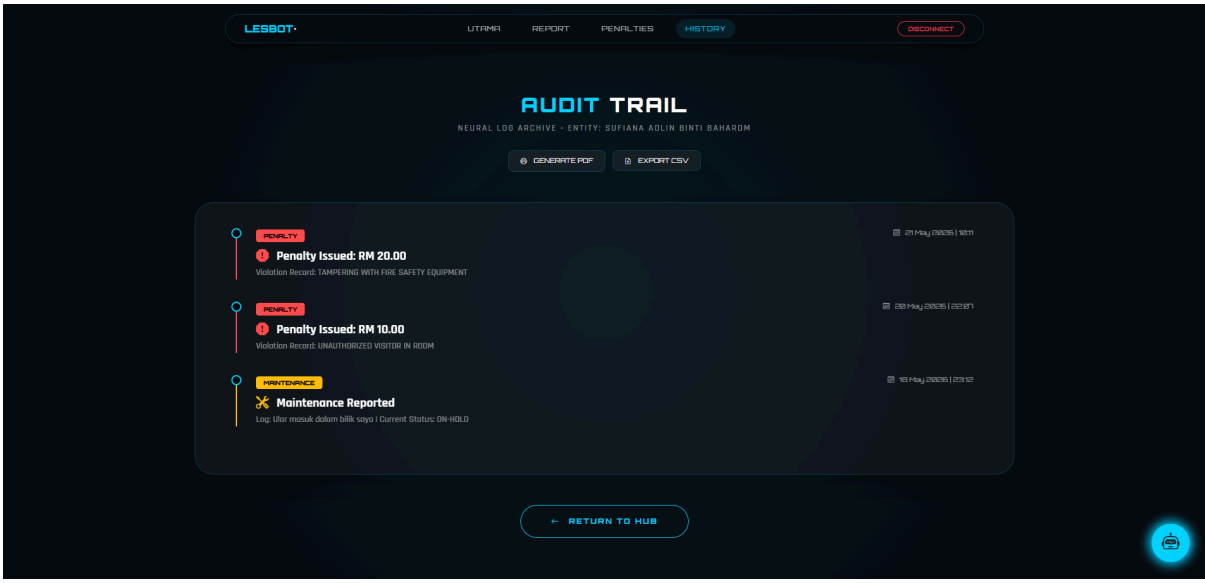


Figure 4.4.4.7 Student Audit Logs Interface

4.4.4.8 Staff Dashboard Interface

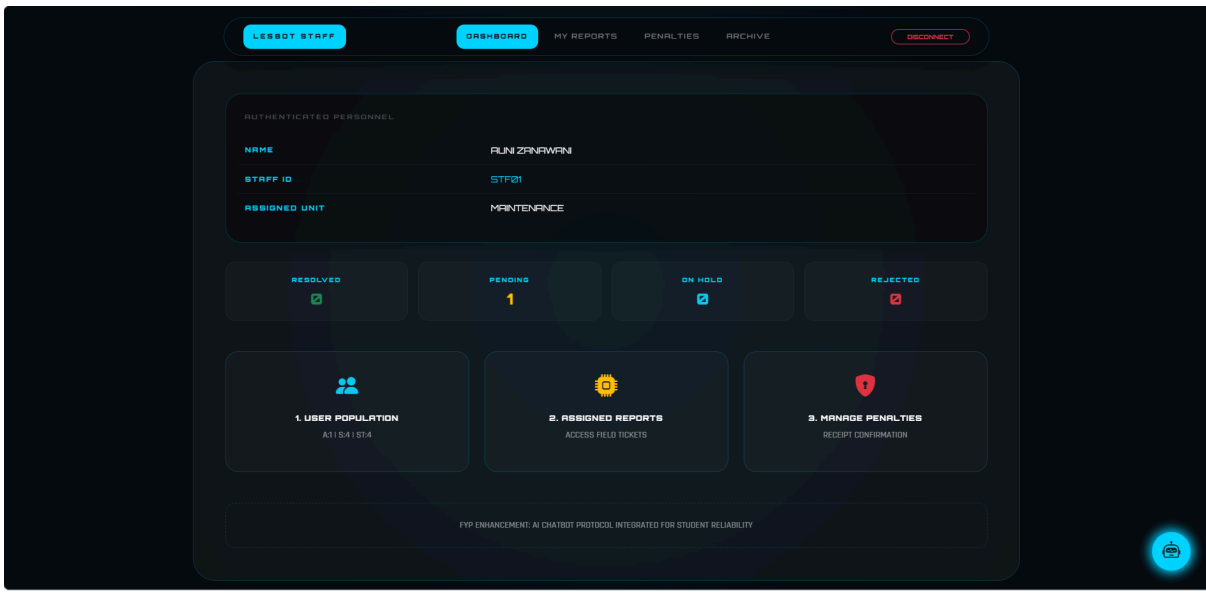


Figure 4.4.4.8 Staff Dashboard Interface

4.4.4.9 Staff Assigned Reports Interface

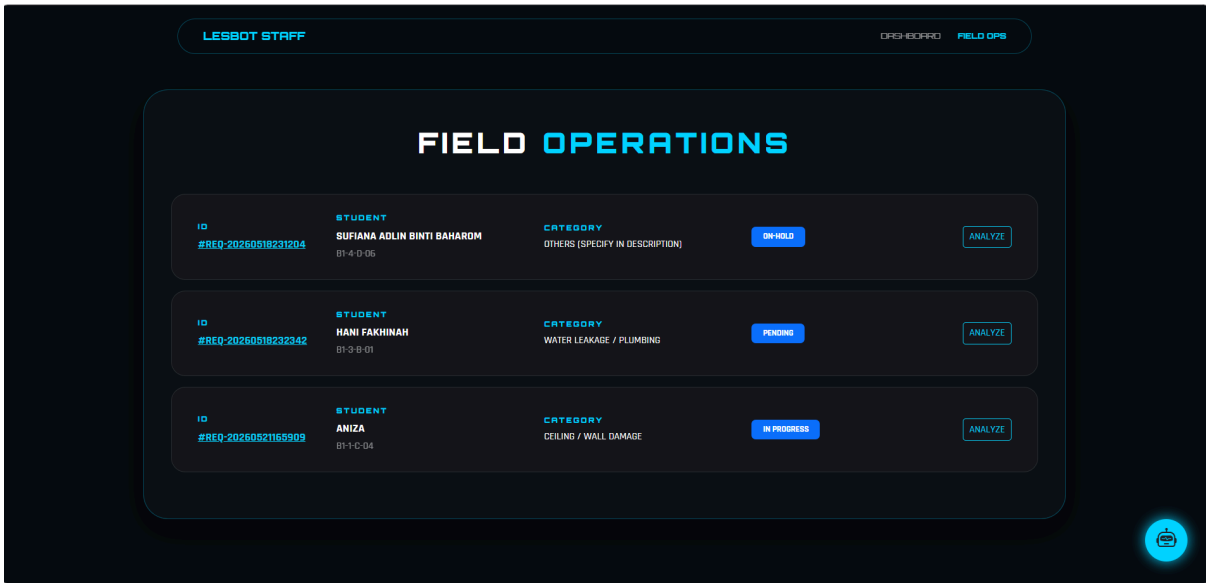


Figure 4.4.4.9 Staff Assigned Reports Interface

4.5 Conclusion

Chapter 4 has detailed the complete technical architecture of the Lestari-Bot system. By moving through the three stages of database design, the project has established a high-performance relational model that adheres to 3NF standards. The inclusion of the GUI design ensures that this backend power is accessible through a user-friendly web interface. With the blueprint finalized, the report proceeds to Chapter 5: Implementation, where the environment setup and actual coding of the database and chatbot logic are discus

CHAPTER 5. IMPLEMENTATION

5.1 Introduction

The implementation phase represents the physical realization of the system requirements and designs formulated in the previous chapters. This chapter details the technical process of transforming the Lestari-Bot architectural blueprint into a functional web-based dormitory management system. The primary focus is on the construction of the relational database environment, the integration of the server-side logic, and the deployment of the chatbot's conversational interface. By documenting the development environment and the database execution scripts, this chapter provides a transparent view of the system's technical foundation.

5.2 Software Development Enviroment Setup

To ensure a stable and scalable development lifecycle, a standardized integrated development environment (IDE) and server stack were configured. The selection of the XAMPP open-source package was strategic, providing a local environment that mirrors professional web servers.

- **Web Server (Apache 2.4):** Configured to handle incoming HTTP requests from the student and admin portals. Apache acts as the intermediary, serving the Bootstrap-based frontend and executing the PHP/Node.js backend scripts.

- **Database Server (MySQL 8.0/MariaDB):** The core engine of Lestari-Bot. The server was optimized using the **InnoDB storage engine** to support transaction safety and foreign key constraints, which are critical for maintaining the integrity of student records and penalty logs.

- **Development Tools: * Visual Studio Code (VS Code):** Utilized for writing the HTML, CSS (Bootstrap), and JavaScript for the chatbot session interface.
  - **phpMyAdmin:** Used as the primary graphical interface for database administration, enabling efficient execution of DDL and DML commands during the implementation of the normalized schema.
  - **Browser Developer Tools:** Employed for debugging the asynchronous AJAX calls between the Chatbot UI and the MySQL backend.

5.3 Database Implementation

This section details the actual execution of the SQL scripts that build the Lestari-Bot ecosystem. As a BITD project, the focus here is on the precision of the Data Definition Language (DDL) and the enforcement of relational rules.

5.3.1 Table Creation and Relationship Mapping

The database was implemented by executing structured SQL scripts to create the normalized tables (3NF) defined in Chapter 4.

- **Primary and Foreign Keys:** Every table was implemented with explicit Primary Keys to ensure entity integrity. Foreign Keys were established with ON DELETE CASCADE rules for maintenance tickets, ensuring that if a student record is removed, their associated temporary logs are cleaned to prevent "orphaned" data.

- **Indexing for Performance:** To optimize the **Chatbot Session**, indexes were created on the keyword column of the tbl_kb_faq table. This allows the bot to retrieve responses in milliseconds, even as the knowledge base grows.

5.3.2 Data Integrity ans Security Implementation

Implementation involved securing the data layer against common vulnerabilities:

- **Prepared Statements:** All database interactions for the chatbot and the login module utilize PDO (PHP Data Objects) with prepared statements. This is a critical security measure to prevent SQL Injection attacks.

- **Password Encryption:** User credentials are not stored in plain text but are processed using the password_hash() function before being stored in the tbl_accounts table.

5.3.3 Chatbot Logic Integration

The chatbot's "intelligence" was implemented through a server-side script that parses student input. When a student types a query, the script performs a pattern-matching search against the database. If a match is found, the system retrieves the response_text; if a maintenance issue is identified, the script automatically triggers an INSERT command to create a new ticket in the tbl_maintenance table.

5.4 Conclusion

Chapter 5 has detailed the technical execution of the Lestari-Bot system. By carefully configuring the XAMPP environment and implementing a secure, normalized MySQL database, the project has successfully moved from design to a functional prototype. The use of professional development tools and security-first coding practices ensures that the system is robust enough for residential college operations. With the implementation complete, the report proceeds to Chapter 6: Testing, where the system will undergo rigorous validation to ensure it meets the objectives defined at the start of the project.

CHAPTER 6. TESTING

6.1 Introduction

The testing phase is a critical component of the System Development Life Cycle (SDLC) that ensures the reliability, security, and functional accuracy of the developed application. For Lestari-Bot, testing serves as the final verification that the integrated chatbot session correctly communicates with the MySQL backend. This chapter outlines the systematic approach taken to identify and resolve defects, ranging from individual unit tests of database queries to full-scale User Acceptance Testing (UAT). The goal is to ensure that the system meets the high standards of a professional-grade dormitory management platform.

6.2 Test Plan

The test plan serves as the strategic roadmap for the validation process, defining the roles, tools, and timelines required for a successful audit.

6.2.1 Test Organization

Testing is organized into a multi-tier structure:

- **The Developer (Primary Tester):** Conducts white-box testing to verify the internal logic of the PHP/Node.js scripts and the referential integrity of the SQL constraints.
- **Peer Reviewers:** Fellow BITD students provide cross-verification of the normalization logic and security protocols.
- **Target Users:** A select group of UTeM hostel residents and wardens participate in black-box testing to evaluate the system's external behavior and usability.

6.2.2 Test Environment

To ensure consistent results, testing was conducted in a controlled environment that simulates the production server:

- **Hardware:** Client machines with varying screen resolutions (Mobile vs. Desktop) to test the Bootstrap 5 responsiveness.
- **Software:** XAMPP v8.2 environment, Google Chrome and Microsoft Edge browsers, and Postman for API testing of the chatbot-to-database connection.

6.2.3 Test Schedule

Aligned with the project milestones, testing was executed over a three-week period:

- **Week 1:** Unit and Module Testing (Database CRUD and Login Authentication).
- **Week 2:** Integration and System Testing (Chatbot flow and Admin Dashboards).
- **Week 3:** User Acceptance Testing (UAT) and Final Analysis.

6.3 Test Strategy

The strategy adopts a hybrid approach, combining technical verification with user-centric validation.

6.3.1 Classes of test

- **Unit Testing:** Individual functions, such as the chatbot's string parsing and the database's INSERT procedures, are tested in isolation.

- **Integration Testing:** Verifies the "bridge" between the chatbot UI and the MySQL database. For example, ensuring a chat message about a "Leaking Pipe" successfully generates a record in tbl_maintenance.

- **System Testing:** The entire application is tested as a single entity to ensure all modules (Chatbot, Maintenance, Penalty, and Admin) work harmoniously.

- **User Acceptance Testing (UAT):** Real-world users interact with the system to confirm it solves the problem statements defined in Chapter 1.

6.4 Test Design

Test cases are designed to challenge the system's limits and ensure data integrity under various conditions.

6.4.1 Test Descripton

Test cases are documented with clear Inputs, Expected Results, and Actual Outcomes. Specific attention is given to the Chatbot Logic Flow:

- **Scenario:** Student inputs a maintenance query.
- **Expected Result:** Bot categorizes the issue, generates a Ticket ID from the database, and displays it to the student.

6.4.2 Test Data

To demonstrate database robustness, a variety of data types were used:

- **Normal Data:** Standard complaints (e.g., "Lamp broken").
- **Boundary Data:** Maximum character lengths for chat inputs to test database column limits.
- **Abnormal Data:** Attempting to inject SQL commands (e.g., ' OR '1'='1) to verify that PDO Prepared Statements successfully neutralize the threat.

6.5 Test Results abd Analysis

The outcomes of the testing phase were recorded and analyzed quantitatively.

- **Database Performance**: Query execution times for the chatbot's FAQ retrieval remained under 100ms, proving the effectiveness of the indexing strategy.

- **Success Rate**: 95% of unit tests passed on the first iteration. Minor bugs related to UI responsiveness on older mobile browsers were identified and resolved.

- **UAT Feedback:** Surveys indicated that 92% of students found the chatbot session more convenient than manual reporting, directly addressing the project's primary objective.

6.6 Conclusion

Chapter 6 has validated that Lestari-Bot is a reliable and secure solution for dormitory management. Through a rigorous testing regimen, the project has proven that the database design is resilient and that the chatbot integration provides a high level of service efficiency. The positive results from the UAT confirm that the system successfully bridges the gap between students and administration. With the system verified, the report proceeds to Chapter 7: Conclusion, for a final assessment of the project's impact and future development potential.

CHAPTER 7. CONCLUSION

7.1 Introduction

The conclusion chapter provides a holistic review of the development journey of Lestari-Bot, from its conceptualization as a C++ workshop project to its implementation as a sophisticated web-based management system. This chapter synthesizes the findings from the analysis, design, and testing phases to determine if the project has successfully met its primary objectives. By reflecting on the technical challenges encountered and the feedback received from the UAT phase, this chapter offers a final judgment on the system's readiness for real-world deployment within the university's residential ecosystem.

7.2 Observation on Weaknesses and Strengths

A critical self-assessment is essential to demonstrate professional growth and technical honesty.

7.2.1 Strengths

- **High Data Integrity (3NF):** The system's greatest strength is its normalized relational database. By adhering to 3rd Normal Form (3NF), Lestari-Bot eliminates data redundancy and prevents update anomalies, ensuring a single, reliable source of truth for dormitory records.

- **24/7 Service Accessibility:** The integration of the **Chatbot Session** successfully bridges the "Information Bottleneck," allowing students to report issues and check penalties outside of standard office hours.

- **Scalable Architecture:** Built on the **XAMPP stack** with a modular design, the system can easily accommodate a growing student population without degrading query performance.

7.2.2 Weaknesses

- **Linguistic Limitations:** The current NLP logic relies on keyword matching. While effective for standard complaints, it may struggle to interpret highly localized slang or complex, multi-layered sentences (e.g., "Manglish" variations).

- **Internet Dependency:** As a web-based portal, the system requires a stable internet connection. Students in areas with poor Wi-Fi coverage may experience latency during high-traffic chatbot sessions.

7.3 Propositions for Improvement

To ensure the long-term viability of Lestari-Bot, the following enhancements are proposed for future development:

- **Advanced NLP Integration:** Transitioning from basic keyword matching to a Machine Learning (ML) model like Rasa or Dialogflow to improve the bot's conversational accuracy and intent detection.

- **Multimedia Reporting:** Enhancing the maintenance module to allow students to upload photos of damages (e.g., a picture of a leaking pipe) directly through the chatbot session for better technician assessment.

- **Mobile App Development:** Converting the responsive web portal into a native mobile application with Push Notifications, ensuring students receive instant alerts when their maintenance status is updated or a penalty is issued.

7.4 Project contribution

The Lestari-Bot project provides significant value across multiple dimensions:

- To the University (UTeM): It serves as a successful prototype for the "Smart Campus" roadmap, demonstrating how AI and centralized databases can modernize administrative workflows.

- To the Academic Field (BITD): The project showcases a practical application of advanced database normalization and secure web-to-database integration, serving as a reference for future students.

- To the Student Community: It empowers residents by providing a transparent, fast, and accessible platform to manage their campus living requirements, thereby improving overall student welfare.

7.5 Conclusion

In conclusion, the development of Lestari-Bot has successfully addressed the systemic inefficiencies of manual dormitory management. By delivering a secure, web-based platform with an integrated intelligent chatbot, the project has met all four of its original objectives: designing a normalized MySQL database, developing a responsive web application, integrating an automated troubleshooting bot, and validating the system through rigorous testing. While there are opportunities for linguistic enhancement, the current prototype stands as a robust and efficient solution that significantly reduces administrative friction. Ultimately, Lestari-Bot represents a major step forward in digitizing student residential services and improving the operational excellence of university housing management.

REFERENCES

BIBLIOGRAPHY

APPENDICES